Fixing the Release: Faithful Post-Hoc Repair of Differentially Private Synthetic Data

Itay Chairman
Hebrew University
itay.chairman@mail.huji.ac.il

Xi He
University of Waterloo
xi.he@uwaterloo.ca

Amir Gilad
Hebrew University
amirg@cs.huji.ac.il

***Abstract*—Differentially private (DP) synthetic data can be shared freely for downstream analysis, but an already generated dataset may still violate integrity constraints or distort statistics that are important for analysis. Regenerating the data is often costly, may require a substantial privacy-budget allocation, and does not preserve the released sample. Existing repair methods, meanwhile, focus on constraint satisfaction and ignore the statistical targets that motivated synthetic data generation.**

We introduce the *synthetic data repair problem*: given a DP synthetic dataset, a set of Denial Constraints (DCs), and specified marginal targets, delete tuples so that all DCs are satisfied while minimizing a weighted objective that trades off data retention against relative marginal error. This soft formulation keeps the problem feasible while capturing the requirements of reliable synthetic data: maintain the largest sample with logical consistency and fidelity to selected private data statistics. We show that the problem is NP-hard even in restricted settings. We then present the first framework for post-hoc repair of DP synthetic data under both integrity and statistical considerations, which complements DP synthesizers: it takes any release as input and corrects it without regeneration. The framework takes marginal targets as input and formulates repair as a mixed-integer linear program that yields an optimal solution. Since exact optimization can be expensive at scale, we also develop a practical two-phase greedy heuristic. Experiments on four real-world datasets and state-of-the-art DP synthesizers show that our approach eliminates DC violations while preserving statistical fidelity, offering a practical tradeoff among integrity, accuracy, and scalability.

I. Introduction

Differential privacy (DP) is widely regarded as the gold standard for protecting individual privacy, offering strong, quantifiable, and composable guarantees for data analysis and release [1], [2]. A particularly powerful application of DP is synthetic data generation. By virtue of the post-processing property of DP [2], DP synthetic datasets can be freely shared and analyzed without additional privacy loss, supporting tasks such as exploratory analysis, model training, and query answering over sensitive data. DP synthesizers, from deep generative models [3]–[5] to statistical approaches [6]–[10], share a common goal of preserving the statistical characteristics of the private data, most notably through the use of noisy low-dimensional marginals, typically two-way marginals, to guide synthetic data generation [7], [8], [11].

In addition to statistical fidelity, synthetic data may be required to satisfy integrity constraints that encode domain knowledge and semantic validity. Denial Constraints (DCs) [12], [13] form the most general and widely adopted class of such constraints, subsuming functional dependencies and other common restrictions by forbidding disallowed patterns across tuples. Recent work has begun to address this need, most notably through Kamino [14], which generates DP synthetic data that satisfies a specified set of DCs.

Synthesizers and repair are complementary. While a synthesizer produces a release that is faithful to the private data on the statistics it measured, the question we address is what to do with that release once it is found to violate constraints or to deviate from important statistics. A straightforward response to inconsistency or bias is to regenerate the synthetic data, but this approach is often prohibitively expensive due to the computational complexity of modern DP synthesis pipelines. Moreover, prior work showed that a released synthetic dataset often needs post-hoc adjustment to meet task-specific demands [15], and the same holds for DCs. Thus, an alternative is to apply existing data repair techniques [13], [16]–[18]. However, classical data repair focuses on enforcing integrity constraints and disregards the statistical objectives that motivated synthetic data generation in the first place, potentially yielding repaired datasets that diverge from the distribution of the private data on specific statistics. Statistics-aware repair methods do not close this gap when applied to the release, since they require data access to measure its statistics [13] or enforce hard bounds on a single attribute [19], whereas after DP synthesis the statistics of the data at hand are rarely identical to the ones from the original private data.

The need for post-hoc repair arises in several concrete scenarios. **(S1) Late-discovered DCs:** Integrity constraints are often identified only after the synthetic data has already been published or distributed, e.g., domain experts may flag violations after the fact. Regeneration requires rerunning the synthesis mechanism, whereas repair is pure post-processing given released marginal targets and incurs additional privacy cost only when new targets must be acquired. **(S2) Computational cost:** State-of-the-art DP synthesizers can require hours of compute (see Section V); repair is orders of magnitude faster. **(S3) Preserving the existing sample:** The released dataset may have been reviewed, annotated, or curated by data stewards; repair emphasizes retaining the original records, preserving the curated sample while fixing violations. **(S4) No access to the private data:** The party that needs the constraints or statistics, e.g., an analyst, holds only the release, so it can neither regenerate the data nor enforce

constraints during generation. In these scenarios, the inputs to repair are typically at hand, namely the release, DCs from the public schema documentation, and marginal targets that marginal-based synthesizers such as AIM [10] and MST [8] already measure privately or that an analyst specifies, so repair is pure post-processing. Generation-time enforcement, e.g., Kamino [14], remains preferable when the DCs and targets are known in advance and the generator scales; repair is the complementary step for the release at hand, applicable to the output of any synthesizer, including constraint-aware ones.

TABLE I: DC violations and relative marginal error for synthetic vs. repaired data for different synthesizers on the Census dataset. Darker red indicates worse values; darker green indicates better values. **Marginal error** reports the average relative error over all selected marginals, measuring how closely the (repaired) synthetic marginals match the marginal targets. (∗) Kamino was run on only 5K tuples due to scalability issues (see Example 1).

Synthesizer	Synthetic Data		Repaired Synth. Data	
	DCs Viol.	Marg. Error	DCs Viol.	Marg. Error
AIM	3.198×10^8	7.985	0	0.247
MST	1.93×10^7	10.576	0	0.45
PATE-CTGAN	1.48×10^{10}	666.34	0	0.754
KAMINO*	0	1.523	0	0.986

Motivated by these gaps, we propose *synthetic data repair*, a post-processing step that sits after any DP synthesizer: given a synthetic dataset, a set of DCs, and specified two-way marginal targets (e.g., noisy marginals privately estimated from the true data), delete tuples so that the result satisfies the DCs while remaining faithful to the targets. We adopt the well-studied deletion-based repair model [17]–[20] and minimize a single weighted loss $\mathcal{L}$ that combines the deletion fraction and the average relative error of the targets, with $\alpha \in [0, 1]$ trading off data retention against marginal fidelity, so the problem is always feasible. We call a minimizer an $\mathcal{L}_\alpha$*-optimal synthetic repair*, where optimality is relative to the deletion model, the given DCs and targets, and α, and we prove the problem is NP-hard even in restricted settings. We give two algorithms, an MILP that yields $\mathcal{L}_\alpha$-optimal repairs but scales poorly, and HYBRID, a two-phase greedy heuristic over the conflict graph [21] that resolves all DC conflicts with the maximum-degree vertex-cover heuristic and then repeatedly deletes the tuple whose removal most decreases $\mathcal{L}$.

Example 1. Consider the Census dataset [22], with sensitive information such as income, education, and nativity. The data needs to satisfy several publicly-known DCs, e.g., that two individuals cannot share a nativity and have different citizenship, and that a state determines its Census division. An agency releases a DP synthetic extract of size 500,000, generated with privacy budget $\epsilon = 0.5$, together with noisy two-way marginals. A health-policy team later obtains the release but has no access to the private microdata (S4). We consider four synthesizers. AIM [10] adaptively measures workload marginals, MST [8] measures marginals along a private tree-structured model, PATE-CTGAN [4] trains a conditional GAN from noisily aggregated teacher votes, and Kamino [14] enforces DCs during synthesis. The team also seeks to preserve specific relevant marginals, such as (race, income) and (education, income), which DP noise may distort [23]. We simulate the released targets by applying the Gaussian Mechanism [24] under zCDP with budget $\rho = 1$ to 50 randomly selected two-way marginals on these attributes (standing in for measurements the agency releases. The private data is hidden from the team itself) whose relative errors (between the repaired marginals and the targets) the repair objective $\mathcal{L}$ aims to minimize directly. Table I reports the DC violations and marginal error per synthesizer, averaged over 100 runs. Kamino, which enforces DCs at generation time, produces no violations, but was run on only 5K tuples due to severe scalability issues (sampling 10K tuples exceeded our 12-hour timeout); moreover, repair still reduces its marginal error from 1.523 to 0.986. Our greedy repair reduces DC violations to 0 and significantly lowers marginal errors, deleting 1.2% of the tuples for MST, 1.3% for AIM, 3.4% for Kamino, and 94.9% for PATE-CTGAN (owing to its extreme violation rate). Thus, using only the released inputs, repair enforces DC compliance and substantially reduces marginal errors on the existing data.

Contributions. This work makes the following contributions:

- **Problem formulation (Section III).** We formalize *synthetic data repair under DP*, the first framework for post-hoc repair of DP synthetic data that jointly enforces DCs and marginal targets, complementing synthesizers rather than replacing them. A single weighted objective trades off deletions against relative marginal error, and the problem is NP-hard even with the marginal terms alone and no DC.
- **Repair algorithms (Section IV).** An exact MILP that finds an $\mathcal{L}_\alpha$-optimal repair via an exact linearization of the size-normalized relative error, and a scalable greedy heuristic that resolves DC conflicts via vertex cover and then deletes tuples along the steepest decrease of $\mathcal{L}$. Both are agnostic to the source of the targets, and satisfy DP by post-processing.
- **Experimental evaluation (Section V).** On four real-world datasets and four DP generators, we repair *organic* synthesis violations and compare against DC-only repair. HYBRID eliminates all violations, scales to large data, and lowers the error on the targeted marginals at a small deletion overhead, with gains that transfer to the true private marginals and a limited effect on global fidelity (Sections V-C and V-D).

We review background in Section II, discuss related work in Section VI, and conclude in Section VII.

II. PRELIMINARIES

We next review the necessary definitions from prior work.

A. Databases, Repairs, and Marginals

We consider a single-relation schema $\mathcal{A}ttr = (A_1, \ldots, A_m)$, where each attribute A_i has a domain $Dom(A_i)$ of values. A database D over $\mathcal{A}ttr$ is a collection of tuples in $Dom(A_1) \times \cdots \times Dom(A_m)$, treated as distinct occurrences (so duplicates can be deleted individually).

ID	Edu-num	Edu.	Occu.	Income	Age	Race
t_1	14	Undergrad	Manager	50K	52	AA
t_2	13	HS	Craft-repair	47K	48	SA
t_3	14	Undergrad	Farming	50K	25	C
t_4	22	Prof_school	Prof	62K	57	AA

(a) Private dataset D_p^E.

ID	Edu-num	Edu.	Occu.	Income	Age	Race
t_1	15	Undergrad	Manager	50K	55	AA
t_2	12	HS	Craft-repair	40K	45	SA
t_3	15	Undergrad	Farming	50K	22	C
t_4	22	Prof_school	Prof	60K	55	AA
t_5	14	Undergrad	Prof	30K	50	AA
t_6	16	Undergrad	Craft-repair	60K	50	AA

(b) Synthetic dataset D_s^E.

Fig. 1: Private and synthetic databases based on population datasets [22], [25].

A database D' is a *subset* of D, denoted $D' \subseteq D$, if it is obtained from D by deleting zero or more tuples. We consider a synthetic database D_s generated, under Differential Privacy, from a private database D_p over the same attribute set $\mathcal{A}ttr$; both are assumed non-empty throughout, and n denotes the data size ($|D_s|$ or $|D_p|$, depending on the context).

Denial Constraints. Following previous work on related topics [13], [20], [21], we focus on Denial Constraints (DCs) on pairs of tuples. Given a database D over a schema $\mathcal{A}ttr$, a DC can be written as $\forall t_1, t_2 \in D,\ \neg(\phi_1 \wedge \ldots \wedge \phi_p)$, where ϕ_i is a predicate of the form $t_i[A_j] \circ v$ or $t_1[A_j] \circ t_2[A_j]$, for $i \in \{1, 2\}$, $A_j \in \mathcal{A}ttr$, $v \in Dom(A_j)$, and $\circ \in \{>, <, \geq, \leq, =, \neq\}$. DCs are a general form of integrity constraint, capturing multiple classes such as Functional Dependencies (FDs), Conditional FDs [26], and metric FDs [27]. If all tuple pairs in D satisfy a set of DCs Σ, we write $D \models \Sigma$. Single-tuple DCs are checked per tuple and repaired by deleting the violating tuples, and DCs over more than two tuples are left for future work. DCs are the constraint language of deployed cleaning systems [13], [28], can be discovered from data [29]–[31], and are supported by private synthesis [14]; their inequality predicates (e.g., salary-to-rate monotonicity in tax data) are beyond FDs. We treat DCs as part of the *public* schema, since they are validity rules over any instance (e.g., codebook relationships), not facts about individuals, obtained from documentation or experts as in [13], [14], [21]. Hence, using them spends no privacy budget, and they may be discovered after generation; mining DCs from the private data would require its own budget.

Database Repairs. Following previous work [17]–[19], we consider repairs via tuple deletions: given a set of DCs Σ, a *consistent subset* of D_s is a database $D_r \subseteq D_s$ such that $D_r \models \Sigma$.

Example 2. Consider the databases D_p^E and D_s^E in Figure 1 based on an income dataset [22], [25]. The database schema is $\mathcal{A}ttr = \{ID, Edu{-}num, Edu., Occu., Income, Age, Race\}$ and it includes $n = 4$ tuples. Consider the following two DCs:

1) $\sigma_1 : \forall t_1, t_2 \neg(t_1[Edu] = t_2[Edu] \wedge t_1[Edu-num] \neq t_2[Edu-num])$

2) $\sigma_2 : \forall t_1, t_2 \neg(t_1[Occu] = \text{'Prof'} \wedge t_2[Occu] = \text{'Craft-repair'} \wedge t_1[Income] < t_2[Income])$

Notice that $D_p^E \models \sigma_1, \sigma_2$ while $D_s^E \not\models \sigma_1, \sigma_2$. Specifically, (t_1, t_5), (t_1, t_6), (t_3, t_5), (t_3, t_6), and (t_5, t_6) violate σ_1, and (t_2, t_5), (t_5, t_6) violate σ_2. A repair of D_s^E would be removing t_5 and t_6.

Marginals. We formalize the notion of 2-way marginals, drawing on the prior work for generating private synthetic data [7], [8], [11].

Definition 3 (2-way marginal)**.** *Given a pair of attributes $A_i, A_j \in \mathcal{A}ttr$, and a dataset D over $\mathcal{A}ttr$, the 2-way marginal of $a_i \in Dom(A_i)$, $a_j \in Dom(A_j)$ in a non-empty D is a normalized count*

$$m_D(i, j) := \frac{1}{|D|} \sum_{t \in D} \mathbb{1}[t[A_i] = a_i \wedge t[A_j] = a_j]$$

Note that the normalization is always by $|D|$, the size of the dataset whose marginal is taken (e.g., $|D_r|$ for a repaired database). For the empty dataset we set $m_\emptyset(i, j) := 0$ by convention (all counts are zero).

B. Differential Privacy

Repair never accesses the private data (Section III); DP enters only through its inputs, the synthetic release and, when measured privately, the marginal targets. We therefore recall only what is needed to read the experiments and defer the rest (the formal definition of zCDP, sequential composition, and the Exponential Mechanism) to Section A. We adopt *zero-concentrated differential privacy* (zCDP) [24], following common practice [8], [10]: a single parameter ρ measures the privacy loss (smaller is more private), the loss of sequential releases adds up, and the guarantee is closed under post-processing, so any computation on released outputs, such as our repair, incurs no further privacy loss. zCDP converts to the classical (ϵ, δ)-DP guarantee [24], [32], and a ρ-zCDP mechanism satisfies $(\rho + 2\sqrt{\rho \ln(1/\delta)}, \delta)$-DP. Two databases $D \approx D'$ are *neighboring* if they have the same size and differ in at most one tuple.

Definition 4 (Gaussian Mechanism [24])**.** *Given a database D, a function $f : \mathcal{D} \to \mathbb{R}^k$ with ℓ_2 sensitivity $\Delta_2 f = \max_{D' \approx D} \|f(D) - f(D')\|_2$, and a privacy parameter $\rho > 0$, the* Gaussian Mechanism *outputs*

$$\mathcal{M}_G(D) \ = \ f(D) + \mathcal{N}\big(0,\, \sigma^2 I_k\big)\,, \qquad \sigma = \frac{\Delta_2 f}{\sqrt{2\rho}},$$

and satisfies ρ-zCDP.

III. Synthetic Data Repair

We now provide the required definitions for our framework and discuss the problem of synthetic data repair.

We first define a relative marginal error, as marginals capture important and relevant statistical properties that we aim for.

Definition 5 (2-way marginal relative error)**.** *Let $\mathcal{A}ttr$ be a schema, $A_i, A_j \in \mathcal{A}ttr$ a pair of attributes, and $(a_i, a_j) \in Dom(A_i) \times Dom(A_j)$ a value pair. A* marginal target *$\tilde{m}(A_i{=}a_i, A_j{=}a_j) \in (0, 1]$ is a specified value for the pair,*

given as input. The 2-way marginal relative error (marginal) *for* $(A_i{=}a_i, A_j{=}a_j)$ *is*

$$E_m(m_{D_r}(A_i{=}a_i, A_j{=}a_j)) = \frac{|m_{D_r}(A_i{=}a_i, A_j{=}a_j) - \tilde{m}(A_i{=}a_i, A_j{=}a_j)|}{\tilde{m}(A_i{=}a_i, A_j{=}a_j)},$$

i.e., the distance between the repaired dataset's marginal and the target, normalized by the target. The error thus depends only on the repaired dataset and the target; a privately obtained target carries its randomness in the mechanism that produced it. Unlike D_p *and* D_s*, the repaired dataset* D_r *may be empty, in which case* $m_{D_r}(A_i{=}a_i, A_j{=}a_j) = 0$ *by Definition 3 and the relative error equals* 1.

Two-way marginals are feasible to estimate under DP and underlie most marginal-based synthesizers [7], [8], [10], so targets can often be reused from a synthesizer's own measurements (e.g., the tree of MST).

Intuitively, the relative error, used by prior work [33]–[35], measures how faithfully the repaired marginal matches the target, normalized so that deviations are penalized proportionally to their target rather than in absolute terms. Rather than a hard constraint, it enters the repair objective as a soft term (Definition 6), keeping the problem always feasible.

For compact notation, we write a marginal error as $E_m(m_{D_r}(i,j))$ and denote the set of monitored value pairs by $\mathcal{M}$.

We now define a synthetic data repair and an $\mathcal{L}_\alpha$-optimal synthetic data repair, drawing inspiration from work on data repair [17], [19].

Definition 6 (Synthetic data repair)**.** *Given a synthetic database* D_s*, a set of marginals* $\mathcal{M}$*, and a set of DCs* Σ*, a synthetic data repair of* D_s *is a consistent subset* $D_r \subseteq D_s$ *satisfying* $D_r \models \Sigma$*. An* $\mathcal{L}_\alpha$-optimal *synthetic repair* $D_r^* \subseteq D_s$ *minimizes the weighted objective*

$$\mathcal{L}_\alpha(D_r) = \alpha \cdot \frac{n - |D_r|}{n} + (1-\alpha) \cdot \frac{1}{|\mathcal{M}|} \sum_{(A_i=a_i,\, A_j=a_j) \in \mathcal{M}} E_m(m_{D_r}(i,j))$$

where $n = |D_s| \geq 1$ *(recall that* D_s *is non-empty) and* $\alpha \in [0,1]$ *balances data retention against marginal fidelity; we write* $\mathcal{L}$ *for* $\mathcal{L}_\alpha$ *when* α *is clear from context.*

Note that $\mathcal{L}$ is well-defined on every subset. In particular, for $D_r = \emptyset$, the convention $m_\emptyset(i,j) = 0$ (Definition 3) gives $E_m(m_\emptyset(i,j)) = 1$ for every monitored marginal, hence $\mathcal{L}(\emptyset) = \alpha + (1-\alpha) = 1$ (and $\mathcal{L}(\emptyset) = \alpha$ when $\mathcal{M} = \emptyset$).

We derive Definition 6 from the classic deletion-based data repair problem, modeled as an optimization problem [17], [18], [20], but incorporate 2-way marginals via a soft weighted objective $\mathcal{L}$. When $\mathcal{M} = \emptyset$, the second term is defined to be 0, and $\mathcal{L}$ reduces to the weighted deletion fraction. The two terms intentionally operate on different scales, encouraging the optimization to prioritize statistical fidelity while still accounting for tuple deletions through the weighting parameter α. The first term is the standard deletion cost of subset repair [17], [19], and the second is the average relative error against the targets (Definition 5). Deleting one tuple changes $\mathcal{L}_\alpha$ by $\alpha/n - (1-\alpha)\,\Delta_{\text{err}}$, where Δ_{err} is the resulting decrease in the average relative error, so a deletion pays off exactly when it lowers the error by more than $\alpha/((1-\alpha)\,n)$. Thus α is the price of a deletion in units of marginal error. When the targets are grossly violated (errors far above 1), this price is negligible and the statistical term rightly dominates; when the errors are small, it is what stops the repair from trading tuples for marginal gains.

Among all consistent subsets, the $\mathcal{L}_\alpha$-optimal repair is the one that minimizes $\mathcal{L}$.

Finally, note that the set of provided marginals $\mathcal{M}$ does not need to contain all 2-way marginals of the private data.

Scope of optimality. $\mathcal{L}_\alpha$-optimality is relative to the deletion model, the given DCs and targets, and α. Importantly, deletions cost no privacy budget (post-processing). A large deletion ratio signals a poor fit between release, DCs, and targets, which α controls (Section V-F). Broader utility is studied empirically in Sections V-B and V-D.

Example 7. Reconsider the databases D_p^E and D_s^E in Figure 1 and the 2 DCs from Example 2. An example of marginals that hold in D_p^E are $E_m(m_{D_r}(Inc{=}50K, Edu{=}Under))$ (large in D_s^E, whose synthetic marginal deviates significantly from the target) and $E_m(m_{D_r}(Edu{=}Under, Race{=}AA))$, where $Under$ is short for $Undergrad$. To get an $\mathcal{L}_\alpha$-optimal synthetic data repair of D_s^E we can remove t_5 and t_6, satisfying both DCs (as shown in Example 2) and minimizing the weighted objective $\mathcal{L}$ from Definition 6, as removing t_5 and t_6 reduces both marginal relative errors while limiting deletions.

Problem definition. We now define the problem addressed in this paper to achieve a framework for synthetic data repair.

Definition 8 (Synthetic data repair)**.** *Given a synthetic database* D_s*, a set of marginal targets* $\mathcal{M}$*, a set of DCs* Σ*, and* $\alpha \in [0,1]$*, find an* $\mathcal{L}_\alpha$*-optimal synthetic data repair for* D_s*.*

All inputs are released or public, so the problem never accesses D_p; targets can be acquired in various ways, and we provide an example DP mechanism for it in Algorithm 6.

Unlike update-based repair that preserves the statistics of the data at hand [13] (Section VI), we aim at *approximating specific statistical properties that may even differ from the current database statistics*, as in Example 1.

However, finding the $\mathcal{L}_\alpha$-optimal synthetic data repair is NP-hard, as we now show.

Hardness of optimal synthetic data repair. Interestingly, the hardness of our problem requires neither both ingredients of $\mathcal{L}$ nor multiple constraints of either type: the marginal terms alone suffice without any DCs (via a reduction from MAX-CUT [36]), a single DC alone suffices even with no marginals (via a reduction from INDEPENDENT SET [36]), and so does the minimal combination of one DC and one marginal. Here the schema, dataset, and marginals are all part of the input, and hardness does not stem from DC repair alone [17]. Hardness

X	B	A_{e_1}	A_{e_2}	A_{e_3}	A_{e_4}
1	0	1	1	0	0
1	0	1	0	1	0
1	0	0	1	1	1
1	0	0	0	0	1
1	i	0	0	0	0

TABLE II: Relation for the reduction of Item (1) of Proposition 9 from MAX-CUT (see Example 10). The first four rows are the vertex tuples ($A_e = 1$ iff the vertex is an endpoint of edge e); the last row stands for the $N = 64$ ballast tuples u_i, $i \in [1, N]$.

of subset repair under a *set* of DCs given in the input is classical [12], and a single FD is tractable [17]; Items (2) and (3) sharpen this to a *single* DC, whose arity grows with the input graph, and Item (1) needs no DC at all.

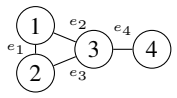
Proposition 9 (Hardness of Definition 8)**.** *The decision version of Definition 8 is NP-hard in each of the following settings:*

1) $\Sigma = \emptyset$*, with the marginal terms of* $\mathcal{L}$ *alone;*
2) $|\Sigma| = 1$ *and* $|\mathcal{M}| = 1$*;*
3) $|\Sigma| = 1$ *and* $\mathcal{M} = \emptyset$*, i.e., a single DC and no statistical constraints.*

We defer the proofs to Section C due to space limitations.

Nevertheless, we illustrate the idea behind Item (1), where hardness comes from the normalization of marginals by the repair size alone. Given a graph $G = (V, E)$ with n vertices, we build a relation with one binary attribute A_e per edge and a *vertex tuple* t_v per vertex, with $t_v[A_e] = 1$ iff $v \in e$, plus $N = 4|E|n$ *ballast tuples* that are 0 on every A_e; we set $\Sigma = \emptyset$, $\alpha = 0$, and give each edge e a target $1/(N + n)$ for the pair $(A_e{=}1, X{=}1)$, where X is a constant attribute. For a repair that retains the vertex set S and all ballast, the normalizing factor $f = (N + n)/|D_r|$ is within $1/(4|E|)$ of 1, so an edge with exactly one retained endpoint ($\kappa_e = 1$) has relative error $f - 1 \approx 0$, while an edge with zero or two retained endpoints has error 1 or $2f - 1 \geq 1$. Hence $|E| \cdot \mathcal{L}(D_r) \approx |E| - \mathrm{cut}(S)$, and minimizing $\mathcal{L}$ is finding a maximum cut. The ballast is what pins f near 1; without it the denominator $|D_r|$ varies with S and the objective no longer tracks the cut, as the example shows.

Example 10. To illustrate Item (1) of Proposition 9, consider the graph G with vertices $\{v_1, v_2, v_3, v_4\}$ and edges $e_1 = (v_1, v_2)$, $e_2 = (v_1, v_3)$, $e_3 = (v_2, v_3)$, $e_4 = (v_3, v_4)$.



Here $n = 4$ and $|E| = 4$, so the construction sets $N = 4|E|n = 64$, $\alpha = 0$, and targets $\tilde{m}(A_e{=}1, X{=}1) = 1/68$, producing the relation in Table II (68 tuples in total). The maximum cut of G has value 3, achieved e.g. by $S = \{v_1, v_2, v_4\}$, which cuts e_2, e_3, e_4. Retaining S together with all 64 ballast tuples gives $|D_r| = 67$ and $f = 68/67$, so the four monitored relative errors are $|2f - 1| = 69/67$ (for e_1, with both endpoints retained) and $f - 1 = 1/67$ (for each of e_2, e_3, e_4), yielding $\mathcal{L}(D_r) = \frac{1}{4}\left(\frac{69}{67} + \frac{3}{67}\right) = \frac{18}{67} \approx 0.269$. For $k = 3$ the threshold is $\lambda = (4 - 3 + \frac{1}{2})/4 = 0.375 \geq \mathcal{L}(D_r)$, certifying a cut of size at least 3; for $k = 4$ we have $\lambda = 0.125$.

We next tackle the repair problem, assuming that the marginals are available (Section IV).

IV. Computing Synthetic Data Repair

We next present two approaches for solving Definition 8: (1) an MILP formulation of the problem that yields an exact ($\mathcal{L}_\alpha$-optimal) solution, and (2) a heuristic two step approach relying on first removing all the violations in the data and then improving the marginals.

A. An MILP Formulation

We next detail an MILP formulation of the problem. Each binary variable x_l represents a tuple $t_l \in D_s$: t_l is retained by the repair iff $x_l = 1$ in the solution to the program. The MILP minimizes the weighted objective $\mathcal{L}$ of Definition 6, a convex combination of the deletion fraction and the average relative marginal error (Definition 5), and we derive it in three steps. Throughout, $n = |D_s|$ is fixed, and for a marginal $m = (A_i{=}a_1, A_j{=}a_2)$ we write $C_m(x)$ for its retained count, so that the repaired marginal is

$$m_{D_r}(A_i{=}a_1, A_j{=}a_2) = \frac{C_m(x)}{\sum_l x_l}, \quad C_m(x) = \sum_{\substack{l:\, t_l[A_i]=a_1,\\ t_l[A_j]=a_2}} x_l, \tag{1}$$

normalized by the *variable* repair size $\sum_l x_l$ (for a non-empty repair; the empty repair is handled in Step 3).

Step 1: capturing the relative errors. For each $m \in \mathcal{M}$ we introduce a continuous variable $d_m \geq 0$ whose intended value is the marginal distance of the repair,

$$d_m \;=\; \left|\, m_{D_r}(A_i{=}a_1, A_j{=}a_2) - \tilde{m}(a_1, a_2) \,\right|. \tag{2}$$

In the objective, d_m is divided by its private estimate $\tilde{m}(a_1, a_2)$ (a positive constant), so the second objective term equals the average relative marginal error $E_m(m_{D_r}(A_i{=}a_1, A_j{=}a_2))$ of Definition 5.

Step 2: exact linearization. Enforcing Equation (2) directly is nonlinear, since by Equation (1) the repaired marginal divides by the variable repair size. Multiplying Equation (2) through by $\sum_l x_l$ yields the pair of constraints

$$d_m \textstyle\sum_l x_l \;\geq\; \pm\big(C_m(x) - \tilde{m}(a_1, a_2) \textstyle\sum_l x_l\big), \tag{3}$$

whose only remaining nonlinearity is the bilinear term $d_m \sum_l x_l$. We linearize it *exactly*: for each marginal m and tuple t_l, a product variable $z_{ml} = d_m \cdot x_l$ is encoded by the standard constraints $z_{ml} \leq d_m$, $z_{ml} \leq x_l$, $z_{ml} \geq d_m - (1 - x_l)$, and $z_{ml} \geq 0$. These are exact because x_l is binary: they force $z_{ml} = d_m$ when $x_l = 1$ and $z_{ml} = 0$ when $x_l = 0$ (in the latter case they also imply $d_m \leq 1$, which excludes no intended solution, since marginals and targets lie in $[0, 1]$). Substituting $\sum_l z_{ml}$ for $d_m \sum_l x_l$ in Equation (3) makes every constraint linear, so the program is a genuine MILP. This step distinguishes our program from ILPs for subset repair under one-way representation bounds [19], which are linear in the

selection variables once multiplied by the repair size; the size-normalized relative error instead requires the products z_{ml}.

Step 3: DCs and the empty repair. DC conflicts are resolved as before with $x_i + x_j \leq 1$ for every conflicting pair t_i, t_j. For the empty repair, $\mathcal{L}(\emptyset) = 1$ (Definition 6); the safeguard constraints $d_m \geq \tilde{m}(a_1, a_2)(1 - \sum_l x_l)$, inactive whenever at least one tuple is retained, force the MILP objective to this exact value at the all-zeros point. Since the marginals appear only in the objective (not as hard constraints), the MILP is always feasible. The complete program is shown in Equation (4). The correctness of the program is established by the following proposition (proof in Section F).

Proposition 11. *Given a synthetic database D_s, a set of DCs Σ, and a set of marginals $\mathcal{M}$, an optimal solution to the MILP in Equation* (4) *induces an $\mathcal{L}_\alpha$-optimal synthetic data repair, i.e., the repair $D_r = \{t_l \mid x_l = 1\}$ minimizes $\mathcal{L}(D_r) = \alpha \cdot (n - |D_r|)/n + (1-\alpha) \cdot (1/|\mathcal{M}|) \cdot \sum_{m \in \mathcal{M}} E_m(m_{D_r}(i,j))$ over all DC-consistent subsets of D_s.*

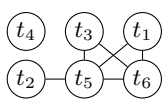
$$
\begin{aligned}
\textbf{min} \quad & \alpha \cdot \frac{n - \sum_{l=1}^{n} x_l}{n} + \frac{1-\alpha}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} \frac{d_m}{\tilde{m}(a_1, a_2)} && (4)\\
\textbf{s.t.} \quad & x_i + x_j \leq 1, \quad \forall\, t_i, t_j \text{ conflicting on } \Sigma \\
& \textstyle\sum_l z_{ml} \geq \sum_{\substack{l:\, t_l[A_i]=a_1,\\ t_l[A_j]=a_2}} x_l \;-\; \tilde{m}(a_1,a_2) \sum_l x_l, \;\; \forall m \in \mathcal{M} \\
& \textstyle\sum_l z_{ml} \geq \tilde{m}(a_1,a_2) \sum_l x_l \;-\; \sum_{\substack{l:\, t_l[A_i]=a_1,\\ t_l[A_j]=a_2}} x_l, \;\; \forall m \in \mathcal{M} \\
& z_{ml} \leq d_m, \quad z_{ml} \leq x_l, \quad z_{ml} \geq d_m - (1 - x_l), \;\; \forall m, l \\
& d_m \geq \tilde{m}(a_1,a_2)\big(1 - \textstyle\sum_l x_l\big), \;\; \forall m \in \mathcal{M} \\
& d_m \geq 0, \quad z_{ml} \geq 0, \quad x_l \in \{0,1\}, \quad \forall\, m \in \mathcal{M},\; l \in [1,n]
\end{aligned}
$$

Note that the size of the MILP can be large: beyond the $|\mathcal{M}|$ distance variables d_m (with $|\mathcal{M}| \leq \binom{|\mathcal{A}ttr|}{2} \cdot \max_{A \in \mathcal{A}ttr} |Dom(A)|^2$), the exact linearization introduces $|\mathcal{M}| \cdot n$ product variables z_{ml} and $O(|\mathcal{M}| \cdot n)$ associated constraints. Together with the $O(|D|^2)$ conflict constraints, the program size is $O(|D|^2 + |\mathcal{M}| \cdot |D|)$. As a result, we show that it encounters scalability issues to get the exact optimum (see Section V). This motivates us to explore a heuristic solution.

B. HYBRID*: A Two-Step Heuristic*

In classic data repair, the objective is to satisfy integrity constraints while maximizing the number of retained tuples. This problem can be reduced to *Vertex Cover* (VC) by constructing a conflict graph [17], [21]. Given a dataset D_s and constraints Σ, the conflict graph $G = (V, E)$ contains one vertex per tuple and an edge between two vertices whenever the corresponding tuples jointly violate a constraint. A vertex cover thus corresponds to a set of tuples whose removal eliminates all violations [20].

Example 12. The DCs in Example 2 induce the following conflict graph for D_s^E from Figure 1b:



Our heuristic (Algorithm 1) addresses the two objectives of Definition 6 in two phases. The first phase deletes tuples until the data is DC-consistent, and the second greedily deletes additional tuples to reduce the distance to the marginal targets. Phase 1 is the standard maximum-degree heuristic for DC repair, whereas Phase 2 is specific to synthetic data repair, since its deletion criterion is the change in the marginal-aware loss, which has no counterpart in DC repair. HYBRID is a heuristic without an approximation guarantee; we quantify its gap to the MILP empirically below.

Phase 1: Resolving conflicts. We first eliminate all DC violations using the standard greedy VC heuristic: while the conflict graph contains edges, we remove the vertex of maximum degree (breaking ties arbitrarily) together with its incident edges (lines 3–5). Removing the highest-degree vertex resolves the largest number of violations per deletion, keeping the repair small. Since DCs are anti-monotone, deleting tuples never introduces new violations, so the phase terminates with a DC-consistent dataset.

Phase 2: Reducing the repair loss. The first phase ignores statistical fidelity. The second phase greedily deletes tuples to reduce the repair loss $\mathcal{L}$ of Definition 6. Starting from the DC-consistent dataset, at each iteration the algorithm removes the tuple whose deletion yields the largest decrease in $\mathcal{L}$ (lines 7–8), stopping when no single deletion reduces it further. Because Phase 2 only deletes tuples, it preserves the DC-consistency established in Phase 1. As an optimization, each iteration ranks the tuples by their hypothetical removal loss and deletes the largest removable prefix of the k best ones as a batch, reducing the number of evaluations of $\mathcal{L}$. Since one deletion changes the normalization of every marginal, the ranking is recomputed after each batch, and a batch is applied only if it decreases $\mathcal{L}$ (otherwise the prefix is shortened down to a single tuple), so the stopping condition of Algorithm 1 is preserved and $\mathcal{L}$ never increases in Phase 2. The complete procedure, with an optimization that efficiently updates the loss function, is given in Section F due to space limitations.

On D_s^E (Example 12), Phase 1 deletes t_5 and then t_6, and Phase 2 deletes nothing, since both targets are then matched exactly (with $\alpha = 0.5$); HYBRID thus returns the $\mathcal{L}_\alpha$-optimal repair of Example 7.

HYBRID satisfies two important properties.

Proposition 13. *For every input, Algorithm 1 (i) returns a repair $D_r \models \Sigma$, and (ii) returns a repair with $\mathcal{L}_\alpha(D_r) \leq \mathcal{L}_\alpha(D'_r)$, where D'_r is its Phase 1 output.*

Item (i) holds since Phase 1 terminates with a consistent subset and Phase 2 only deletes tuples, and DCs are anti-monotone; Item (ii) holds since $\mathcal{L}_\alpha$ decreases strictly in every Phase-2 step. The novel Phase 2 greedily triangulates target adherence and deletion minimization.

Design alternatives. The two-phase structure of Algorithm 1 was chosen against three natural alternatives, each of which we implemented and measured. Replacing Phase 1 with an *optimal* DC repair (or any DC-only algorithm) barely reduces

Algorithm 1: HYBRID: Two-Step Synthetic Data Repair

input : D_s, conflict graph $G = (V, E)$ based on D_s and Σ, α, and a set of marginals $\mathcal{M}$.
output: Repaired dataset D_r.

$D_r \leftarrow D_s$;
/* Phase 1: resolve DC conflicts via greedy VC. */
while $E \neq \emptyset$ **do**
  $v^\star \leftarrow \arg\max_{v \in V} \deg(v)$;
  $D_r \leftarrow D_r \setminus \{t_{v^\star}\}$;
  Delete $v^\star$, its incident edges, and any resulting isolated vertices from G; update V, E;
/* Phase 2: greedily reduce the loss $\mathcal{L}$ (Definition 6). */
while $\exists\, t \in D_r :\ \mathcal{L}(D_r \setminus \{t\}) < \mathcal{L}(D_r)$ **do**
  $t^\star \leftarrow \arg\min_{t \in D_r}\ \mathcal{L}(D_r \setminus \{t\})$;
  $D_r \leftarrow D_r \setminus \{t^\star\}$;
return D_r;

deletions over the maximum-degree heuristic yet is NP-hard and does not scale (Section V). Replacing Phase 2 with the MILP of Proposition 11 drops the $O(|D|^2)$ conflict constraints, which are the main obstacle to solving the full MILP, and is therefore solvable at this size, but lowers marginal error only slightly at a prohibitive cost: on Census data from AIM it improves the error from 0.23 to 0.09 at nearly identical deletions ($\approx 1.3\%$), but takes over 6 hours versus roughly 8 seconds for HYBRID. Collapsing both phases into a single *weighted* vertex cover, with each vertex weighted by the loss decrease of its hypothetical removal, produced the same deletion ratio and marginal error as Phase 1 alone in our runs: minimizing a cover's weight does not minimize the final loss, and a cover-based method has nothing to delete when the conflict graph has no edges.

Complexity analysis. The complexity of Algorithm 1 is $\mathcal{O}(|\mathcal{M}|n^2)$, where $n = |V|$. Phase 1 performs at most n iterations, each removing a single vertex, for $\mathcal{O}(n^2)$ in total. In Phase 2, the effect of a deletion on $\mathcal{L}$ can be evaluated incrementally in $\mathcal{O}(|\mathcal{M}|)$ per tuple (details in Section F), so each of the at most n iterations costs $\mathcal{O}(|\mathcal{M}|n)$, giving $\mathcal{O}(|\mathcal{M}|n^2)$ overall; with batches of size k, as part of the optimization detailed in Section F, the number of Phase 2 iterations drops to at most n/k in the common case, which is what makes the runtimes of Section V-C attainable. For k-way targets, matching a tuple against a target costs $\mathcal{O}(k)$, giving $\mathcal{O}(k|\mathcal{M}|n^2)$.

V. EXPERIMENTS

We perform an experimental evaluation of our repair approach, aimed at answering the following questions: (1) How does our approach perform in terms of repair cost and marginal error compared to existing baselines? (2) How does our approach affect the faithfulness of the synthetic data to the private data compared to existing baselines? (3) How does our approach scale compared to the baselines? (4) What is the effect of the tradeoff parameter α on the performance of Algorithm 1?

Summary of our findings. HYBRID (Algorithm 1) eliminates *all* DC violations, up to 9.5×10^{10} violations, for every dataset and generator, and reduces the error on the targeted marginals in 15 of 16 configurations, by up to nearly five orders of magnitude, while typically deleting under 10% of the tuples (for AIM and MST); global fidelity deteriorates by at most 24% (two-way TVD, Section V-B). Relative to the baselines, it trades slightly more deletions for substantially lower marginal error at a moderate runtime cost (Section V-C), whereas the exact MILP exceeds our four-hour limit on large instances (Section V-E). The accuracy improvements on the noisy targets transfer to the *true* private marginals at every target budget we test, down to below the zCDP equivalent of the generation budget in most configurations (Section V-D), and performance is robust to the choice of the trade-off parameter α (Section V-F).

A. Experimental Setup

All experiments were conducted on a machine with 8 CPU cores and 64 GB RAM using Python 3.11. We used Pandas and DuckDB for data processing and Gurobi to solve the MILPs.[1] To avoid a memory issue, we used a self-implementation of the conflict graph. Unless stated otherwise, each experiment was repeated using 10 independently generated synthetic datasets and 10 independently sampled noisy marginal sets, resulting in 100 runs per configuration.

Datasets. We evaluate on four real-world datasets commonly used in data cleaning and synthetic data generation.

- **IPUMS-CPS (Census)** [22]: 3,395,604 tuples and 8 attributes (6 categorical and 2 bucketized numerical attributes) from the 2022 U.S. Census Population Survey.
- **Adult** [25]: 48,842 tuples and 15 demographic attributes derived from the U.S. Census.
- **COMPAS** [37]: 60,798 tuples and 17 attributes describing criminal history and recidivism.
- **Tax** [29]: 999,910 tuples and 9 demographic and tax-related attributes.

Numerical attributes were bucketized and identifier or near-identifier attributes (e.g., IDs) were removed.

Marginal targets. We evaluate repairs conditional on a supplied set of statistical targets. Unless stated otherwise, each run uses 50 two-way marginals sampled uniformly, and measured using the Gaussian mechanism with total budget $\rho = 1$. Since repair takes the targets as an input, they may come from any source, such as an analyst's preferences, a marginal release by the curator, or statistics published by the synthesizer at a separate occasion; our Gaussian measurements simulate one such release; the repair itself never accesses the private data, and an analyst as in Section I at (S4) would reuse released measurements rather than take new ones. The generators run

[1] Source code: https://github.com/ItayChairmanHuji/SyntheticDataRepair

TABLE III: Example denial constraints (DCs) and their intuition. Universal quantifiers are omitted for brevity.

Dataset	DC	Intuition
IPUMS-CPS	$\neg(t.AGEP = 0 \wedge s.AGEP = 0 \wedge t.SCHL \neq s.SCHL)$	Newborns below 2 years old must have the same education level.
Adult	$\neg(t.capital_gain < s.capital_gain \wedge t.capital_loss < s.capital_loss)$	Capital gain and loss cannot both increase.
Compas	$\neg(t.DecileScore = s.DecileScore \wedge t.ScoreText \neq s.ScoreText)$	Equal scores must have equal labels.
Tax	$\neg(t.State = s.State \wedge t.Salary > s.Salary \wedge t.Rate < s.Rate)$	Within a state, higher salaries cannot have lower tax rates.

at $\epsilon = 0.5$, i.e., $\rho \approx 0.005$ (Section II-B). Section V-D sweeps ρ from 1 down to 0.004, slightly below the generation budget, to measure how much of the gain depends on target accuracy.

We sample each target set once and reuse the same noisy measurements across all methods. We generate 10 independent sets using different seeds and repeat every experiment on each set; each represents a separate $\rho = 1$ execution rather than a joint release. When varying the number of marginals, we use nested prefixes of the same ordered sample and keep the total budget fixed at $\rho = 1$, isolating the effect of additional targets from sampling variability and privacy expenditure. Random targets also decouple repair quality from target selection.

Denial constraints (DCs). Most DCs are taken from previous work [13], [19], [38]. A representative DC for each dataset used in our experiments is provided in Table III. For the sake of the experiments, additional constraints were identified after bucketization and exposed new deterministic relationships between attributes (e.g., the first row in Table III). DCs introduced by bucketization encode realistic rules that every valid record satisfies, and they are applied identically to every method. The complete constraint set is given in Section J.

Synthetic data generators. We evaluate our repair methods on synthetic datasets generated using state-of-the-art DP generators, including PATE-CTGAN [4], AIM [10], MST [8], and Kamino [14]. Evaluating on the output of these generators lets us test our approach on data with *organic* constraint violations and statistical distortions, those arising naturally from the synthesis process, exercising the system under realistic conditions. Unless stated otherwise, all generators use their default hyperparameters, privacy budget $\epsilon = 0.5$, and generate datasets with 500K tuples. As Kamino was not scalable to support large datasets, it was trained on a sample of 5K tuples of the data and generated synthetic data with 5K tuples; its violation counts and error ratios are therefore not directly comparable to those of the 500K-tuple releases (violating pairs grow quadratically with size), and we report them for completeness.

Repair methods and baselines. We evaluate the proposed HYBRID repair algorithm (Algorithm 1) with a batch size of 100 tuples for deletion in Phase 2 and $\alpha = 0.5$; as Section V-F shows, repair quality is robust to the choice of α, varying only modestly across its range. We have compared HYBRID to the MILP (Section IV-A) in Section V-E showing that the MILP approach is infeasible in most cases. Therefore, we focus on examining the performance of HYBRID in this section. We compare against the deletion-based repairs that support DCs alone, which isolates the effect of the marginal-aware phase:

- **OPTIMAL-DC-BASELINE** [20]: the optimal ILP minimizing tuple deletions while ignoring marginal information.
- **CLASSIC-VC-BASELINE** [39]: the standard 2-approximation obtained by reducing data repair to the vertex cover problem on the conflict graph.
- **VANILLA-VC-BASELINE** [40]: the unweighted greedy vertex cover algorithm, which repeatedly removes the tuple participating in the largest number of violations.

Evaluation metrics. We evaluate both repair cost and data quality using three metrics.

- **Deletion ratio:** The fraction of tuples removed during repair (first component in the loss function in Definition 6).
- **Relative marginal error:** As defined in Definition 5.
- **Two-way TVD:** The average Total Variation Distance (TVD) over all 2-way marginals between the repaired and private datasets.

B. Overall Evaluation

We give an overview evaluation of Algorithm 1: each generator produces a synthetic dataset under its default configuration, which we then repair. Table IV summarizes repair quality along four axes: DC violations, marginal error, deletions, and global fidelity.

Repair is necessary even alongside constraint-aware generation. The first column group of Table IV shows that AIM, MST, and PATE-CTGAN produce between 10^7 and over 10^{10} violating pairs on every dataset. Kamino, which enforces DCs at generation time, attains zero violations on three of the four datasets but still produces 1.56×10^5 on Tax, and, unlike post-hoc repair, requires the DCs *before* synthesis (Scenario (S1) in Section I); it also ran on only 5K tuples due to scalability issues, timing out at 12 hours even for 10K tuples. In every configuration, Algorithm 1 reduces the violation count to zero.

Consistency comes with a marked improvement in the target statistics and, in most settings, a modest deletion cost. The second column group reports the ratio of mean marginal error after repair to before (below one indicates improvement): repair lowers it in 15 of 16 configurations, often by several orders of magnitude; the sole exception is Tax under PATE-CTGAN, where the error rises by roughly 44.5%. Repair improves even Kamino's marginals (ratios of 0.81–0.88), showing that generation-time DC enforcement does not substitute for statistical repair, and that the two are complementary (Kamino's 5K-tuple scale makes its counts incomparable to the other columns, see Section V-A). For deletions (third group), AIM and MST lose a small fraction of tuples, typically below 10% and never more than 30%, whereas PATE-CTGAN requires deleting almost the entire dataset; as Section V-C shows, this is not an artifact of our algorithm but a consequence of the extreme violation counts, and deletion-minimizing classical repairs behave similarly. Repair should be used when the generated data is salvageable, as for AIM and MST, whereas data generated by PATE-CTGAN, where 93–99% of the tuples participate in violations,

TABLE IV: Overall evaluation of HYBRID. The table reports the number of denial constraint (DC) violations before and after repair, the ratio of mean marginal error after repair (repaired/synthetic), the deletion ratio, and the ratio of two-way TVD after repair (repaired/synthetic). For the ratio columns, values below one indicate improvement, while values above one indicate deterioration. (∗) Kamino was run on only 5K tuples due to severe scalability issues.

Dataset	DC Violations					Marginal Error Ratio (Repaired / Synthetic)				Deletion Ratio				Two-way TVD Ratio (Repaired / Synthetic)			
	AIM	KAMINO*	MST	PATE CTGAN	After Repair	AIM	KAMINO*	MST	PATE CTGAN	AIM	KAMINO*	MST	PATE CTGAN	AIM	KAMINO*	MST	PATE CTGAN
Adult	1.19×10^{10}	0	6.21×10^{9}	7.18×10^{10}	0	0.0011	0.8075	0.0013	0.0005	0.081	0.0111	0.069	0.998	1.0002	1.133	1.053	1.113
Census	3.20×10^{8}	0	1.93×10^{7}	1.48×10^{10}	0	0.195	0.839	0.414	0.0078	0.013	0.034	0.012	0.949	1.123	1.008	1.052	1.052
Compas	3.92×10^{8}	0	7.78×10^{8}	9.51×10^{10}	0	0.0024	0.881	0.172	2.2×10^{-5}	0.029	0.0055	0.034	0.975	1.064	1.0001	1.067	0.958
Tax	1.20×10^{9}	1.56×10^{5}	7.21×10^{8}	4.76×10^{9}	0	0.984	0.883	0.701	1.445	0.295	0.5479	0.202	0.990	1.241	1.0062	1.244	1.086

is inconsistent beyond repair and should be regenerated to comply with the DCs; the deletion ratio of a DC-only repair thus also serves as a diagnostic of the synthetic data itself. For Kamino on Tax, the 55% deletion ratio is driven primarily by the loss-reduction phase rather than conflict resolution, consistent with its comparatively small violation count.

Beyond the targeted marginals, repair largely preserves global statistical structure. The final column group reports the ratio of two-way TVD to the private dataset after versus before repair. Since Algorithm 1 optimizes only the monitored marginals, tightening them can perturb the remaining distributions; the resulting TVD ratios grow by at most 24%, and in one configuration (Compas, PATE-CTGAN) global fidelity even improves. Overall, repair substantially improves the target statistics while largely retaining the global statistical properties of the synthetic data.

C. Repair Quality and Scalability Analysis

We evaluate the repair algorithms along three dimensions: repair cost (deletion ratio), statistical utility (marginal error), and runtime, as a function of the generator's privacy budget ϵ and the number of marginals, with runtime measured separately against synthetic dataset size on Census. The repair-quality experiments are reported on three of the four datasets (Census, Compas, and Tax); Adult, a subset of Census whose behavior mirrors it, is deferred to Section J for space. Figure 2 compares the methods at the default setting ($\epsilon = 0.5$, 50 marginals). Each row corresponds to one dataset and generator. The left panel divides the marginal error after repair by the marginal error of the unrepaired synthetic data, so points left of the dashed line at 1 are improvements, and the horizontal distance between HYBRID (star) and the DC-only baselines measures the benefit of the marginal-aware phase. The right panel shows the deletion ratio of each method. Figure 3 varies ϵ (Figure 3a) and the number of marginals (Figure 3b). Its top rows report the same error ratio, and its bottom rows report the deletions of each method in excess of the best DC-only repair (VANILLA-VC-BASELINE or OPTIMAL-DC-BASELINE), in percentage points. Figure 3 shows MST, which we chose as the representative generator because it produced good synthetic data overall (low violation counts and small deletion ratios in Table IV), so the trends reflect the repair methods rather than defects of the release; each point is averaged over the synthetic and marginal seeds, and the corresponding plots for AIM, PATE-CTGAN, and Kamino

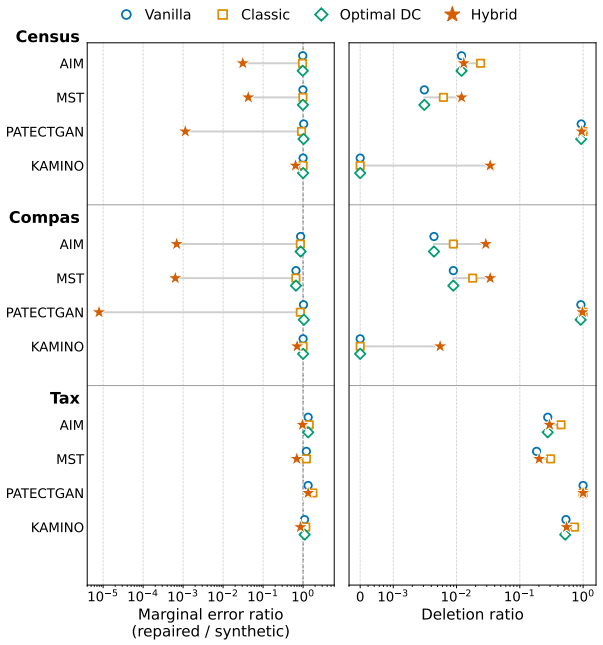
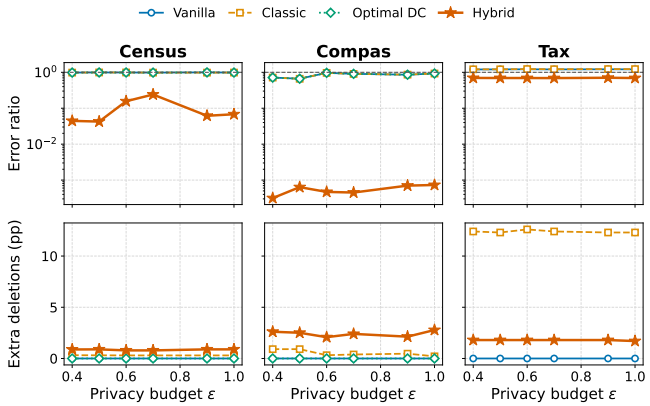


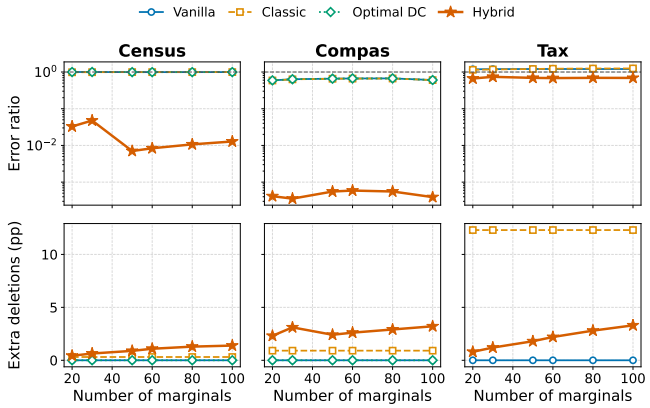
Fig. 2: Repair quality at the default setting ($\epsilon = 0.5$, 50 marginals). Each row is a dataset and generator. Left, marginal error after repair divided by that of the unrepaired synthetic data (log scale, values below 1 are improvements). Right, deletion ratio (log scale). A missing OPTIMAL-DC-BASELINE marker indicates a 12-hour timeout.

are given in Section J. Normalizing by the unrepaired data makes configurations whose absolute errors differ by orders of magnitude comparable. Absolute per-configuration values are given in Section J.

Deletion ratio. The right panel of Figure 2 and the bottom rows of Figure 3 report the fraction of tuples removed during repair. As expected, OPTIMAL-DC-BASELINE achieves the lowest deletion ratio, while VANILLA-VC-BASELINE closely matches the optimum. Since HYBRID additionally optimizes statistical fidelity, it performs slightly more deletions, whereas CLASSIC-VC-BASELINE often removes substantially more tuples because it ignores the topology of the conflict graph. For example, in Census AIM, HYBRID removes 1.3% of the data while CLASSIC-VC-BASELINE removes 2.4% of the data and OPTIMAL-DC-BASELINE and VANILLA-VC-BASELINE remove 1.2%. The extra deletions of every method are nearly constant in ϵ (Figure 3a); for MST, the overhead of HYBRID



(a) Varying the generation privacy budget ϵ (50 marginals).



(b) Varying the number of marginals ($\epsilon = 0.5$).

Fig. 3: Sensitivity of repair quality. Top rows, marginal error ratio as in Figure 2. Bottom rows, deletions in excess of the best DC-only repair (VANILLA-VC-BASELINE or OPTIMAL-DC-BASELINE), in percentage points. Results are for MST, chosen as a representative generator. The other generators are reported in Section J. A missing OPTIMAL-DC-BASELINE marker indicates a 12-hour timeout.

over the best DC-only repair stays below 1 percentage point on Census and below 3 on Compas and Tax, compared with up to 12.6 for CLASSIC-VC-BASELINE on Tax. Figure 3b shows that increasing the number of marginals affects only HYBRID, since the remaining algorithms do not optimize statistical utility. In most settings, additional marginals result in a modest increase in the deletion ratio. In particular, for MST, from 0.4 to 1.4 percentage points over the best DC-only repair on Census and from 0.8 to 3.3 on Tax as the number of marginals grows from 20 to 100.

Marginal error. The left panel of Figure 2 and the top rows of Figure 3 evaluate the statistical fidelity of the repaired datasets. In Figure 2, the DC-only methods cluster around the dashed line (median ratio 1), i.e., they leave the marginal error largely unchanged and sometimes increase it, e.g., on Tax. HYBRID achieves the lowest error among the repair methods in every configuration (tied with VANILLA-VC-BASELINE on Tax with PATE-CTGAN), by up to nearly five orders of magnitude on Census and Compas. On Tax, the gains are smaller (ratios

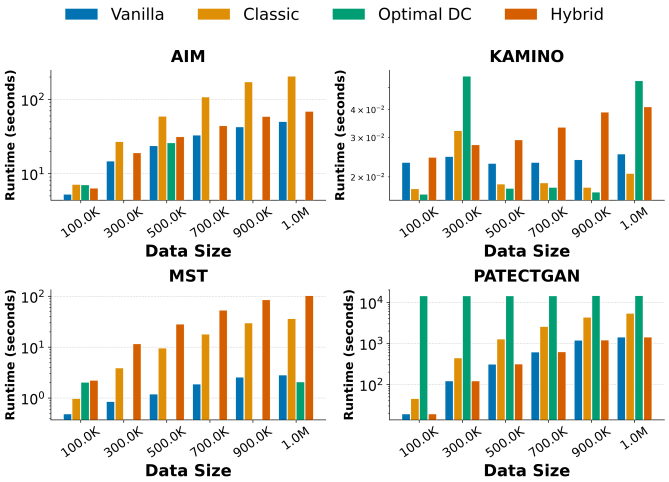


Fig. 4: Runtime as a function of the synthetic data size for the Census dataset.

of 0.694 to 0.967), and for PATE-CTGAN HYBRID matches VANILLA-VC-BASELINE without improving on the synthetic data. The relative improvement of HYBRID is largely stable across ϵ and the number of marginals (Figure 3). For MST the error ratio stays between 3×10^{-4} and 8×10^{-4} on Compas and between 0.67 and 0.74 on Tax, and on Census it stays below 0.07 except at $\epsilon \in \{0.6, 0.7\}$, where it rises to 0.16 and 0.24. There, HYBRID's absolute error after repair is 1.12 and 1.59 instead of $\sim$0.44, i.e., the greedy Phase 2 stopped at a worse local optimum.

Runtime. Figure 4 reports the runtime of the repair algorithms on the Census dataset for different synthetic data generation methods and dataset sizes. Runtime is governed by the size of the conflict graph rather than by $|D|$ alone. On Census with AIM (3.2×10^8 violating pairs and 1.3% deletions) HYBRID finishes in about 8 seconds at 500K tuples, whereas PATE-CTGAN releases with over 10^{10} violating pairs, almost all of whose tuples Phase 1 deletes, take minutes. Runtime increases with the size of the repaired dataset, as larger datasets contain more tuples and typically induce more denial constraint violations. Among the heuristic methods, HYBRID incurs only a modest overhead over VANILLA-VC-BASELINE as it additionally evaluates the statistical objective after each candidate deletion. Although OPTIMAL-DC-BASELINE solves an NP-hard optimization problem (Proposition 9), it remains competitive on several instances due to the effectiveness of modern ILP solvers. However, consistent with its worst-case complexity, several large instances did not converge for over four hours and were therefore terminated. Overall, HYBRID provides substantially improved statistical utility while maintaining runtime comparable to existing scalable repair algorithms. For example, in PATE-CTGAN repairing a data with 1M tuples takes around 3 hours for OPTIMAL-DC-BASELINE around 1.5 hours for CLASSIC-VC-BASELINE and around 15 minutes for VANILLA-VC-BASELINE and HYBRID.

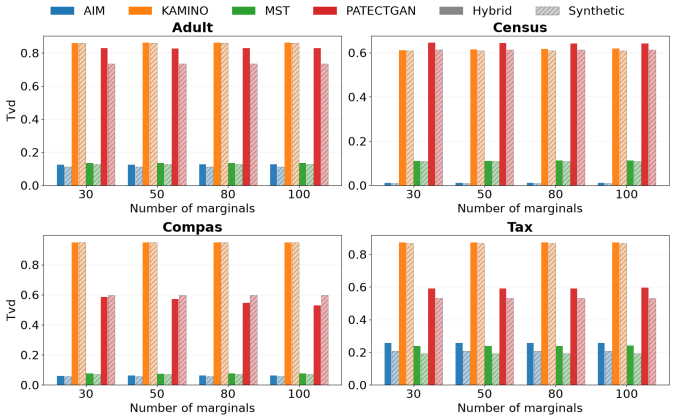


Fig. 5: Two-way TVD as a function of number of marginals. The plots show that HYBRID usually matches the TVD of the synthetic data, and never drastically increases it.

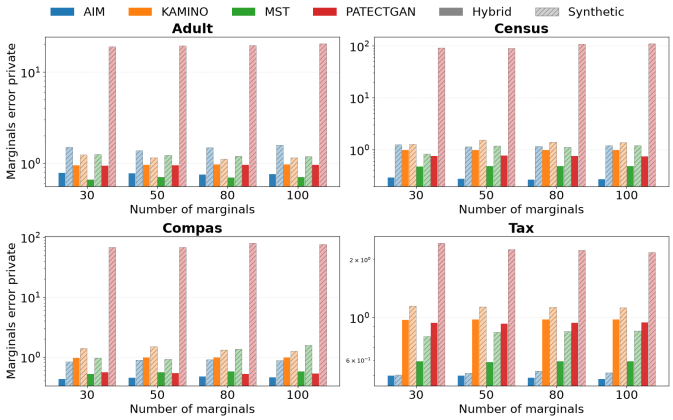


Fig. 6: Marginal error with respect to the *marginals on the private data* as a function of the number of marginals. The graphs show that the marginal error compared with the private data marginals improves significantly from the starting synthetic data after the repair.

D. Faithfulness to the Private Data

This section evaluates whether HYBRID improves the similarity of the repaired data to the original private dataset by comparing it *directly* to the private data, rather than to the noisy marginals supplied during repair as in the previous experiments. Specifically, we report the average two-way TVD and the marginal error with respect to the true private marginals, comparing HYBRID against the original synthetic dataset to illustrate the effect of the repair.

Two-way TVD. Figure 5 compares the average two-way TVD before and after repair across all datasets and generation methods. The repair process has only a modest effect on the total TVD of the synthetic data. Although HYBRID modifies the dataset to satisfy the DCs and improve the selected marginals, the overall pairwise distribution remains largely unchanged. This indicates that the repair process preserves the synthetic data's global statistical properties while enforcing consistency. Figure 14 presents the TVD as a function of the privacy budget ρ for marginal generation. The TVD remains mainly unaffected by changes in the budget. This makes sense as the set of marginals considered by the repair is a small subset of all two-way marginals, so even a large change in their error moves the global average only slightly.

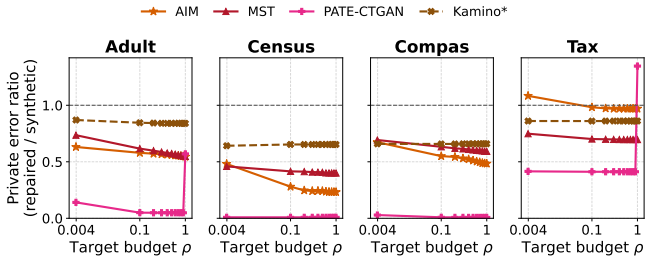


Fig. 7: Error with respect to the true private marginals after repair by HYBRID, divided by that of the unrepaired synthetic data, as a function of the budget ρ spent on the targets. Only HYBRID depends on ρ; the DC-only baselines ignore the targets. Values below the dashed line are improvements; the leftmost point, $\rho \approx 0.004$, lies slightly below the zCDP equivalent of the generation budget ($\rho \approx 0.005$).

Marginal error with respect to the private data. Figure 6 reports the marginal error with respect to the true private marginals. Across nearly all datasets and generation methods, HYBRID consistently reduces the marginal error compared to the original synthetic data. This result demonstrates that optimizing noisy marginals successfully transfers to the underlying private distribution. Although the repair algorithm has access only to DP statistics, the repaired datasets capture the true statistics of the private data. Together with the TVD results, these experiments show that HYBRID substantially improves statistical fidelity while introducing only minor changes to the overall data distribution.

Effect of the target budget. Since the targets in the previous experiments are measured at $\rho = 1$, we next ask whether the transfer survives noisier targets, down to $\rho \approx 0.004$, slightly below the zCDP equivalent of the generation budget (Figure 7). Only HYBRID is affected by ρ, and we report its error relative to the unrepaired synthetic data. The improvement over the synthetic data holds at every budget. At $\rho = 0.1$ the ratio is 0.55 to 0.65 for AIM and MST on Adult and Compas, 0.28 to 0.49 on Census, 0.65 to 0.86 for Kamino, and below 0.06 for PATE-CTGAN on Adult, Census, and Compas. At $\rho \approx 0.004$, where the targets are as noisy as the synthesizer's own measurements, it is still 0.46 to 0.75 for AIM and MST, 0.64 to 0.87 for Kamino, and 0.01 to 0.41 for PATE-CTGAN, in every configuration but Tax with AIM (1.08), where the synthetic data already matches the private marginals. The exception is PATE-CTGAN at $\rho = 1$ on Adult and Tax (0.57 and 1.35, against 0.05 and 0.41 at every other budget): Phase 1 leaves a few hundred tuples there, on which the greedy Phase 2 stopped early (on Tax without deleting any tuple), the local-optimum effect noted in Section V-C. From 0.004 to 0.1 the ratio drops by up to 0.20 (Census with AIM), and from 0.1 to 1 by at most 0.07: most of the improvement is already available from moderately noisy targets. On Tax,

the DC-only repairs *increase* the error (e.g., from 0.51 to 0.68 for AIM), whereas HYBRID keeps it at the synthetic level, so there the marginal-aware phase recovers what constraint enforcement alone loses. In practice this means a curator need not spend a large budget on the targets. Measuring them at the generator's own budget already yields most of the gain, and the remaining budget is better spent on synthesis.

E. HYBRID *Compared to the MILP*

We compare HYBRID to the MILP of Section IV-A on Adult releases from AIM, MST, and PATE-CTGAN ($\epsilon = 0.5$, $\alpha = 0.5$), varying the release size from 1K to 10K tuples with 50 targets and the number of targets from 20 to 100 at 5K tuples, repeating each configuration over independently generated releases and target sets where feasible, with a 12-hour limit per run. The MILP exceeded 4 hours on 138 of its 196 runs (70%) and reached a proven optimum in only 58; at 3,000 tuples and above, every run hits the time limit and the solver returns its incumbent, the trivial solution. At 5,000 tuples the MILP averages $\sim$11,600 seconds and a 0.7 deletion ratio against HYBRID's $\sim$0.2 seconds and 0.39, while its marginal error is only slightly lower ($\sim$0.6 versus $\sim$0.73 on the instances it solves), an edge bought by deleting far more tuples and running for hours.

F. Sensitivity to the Tradeoff Parameter α

The tradeoff parameter α balances data utility and statistical utility in the objective of HYBRID (Definition 6). Since HYBRID is the only repair algorithm that optimizes both objectives simultaneously, it is also the only method affected by α. Figure 13 shows that increasing α consistently reduces the deletion ratio while slightly increasing the marginals error, confirming that α provides the intended tradeoff between the two objectives. For example, in Census AIM, when $\alpha = 0$ the deletion ratio is around 8% of the data and the marginals error is 0.222, as α increases the deletion ratio decreases to 1.3% of the data and the marginals error increases to 0.244. Importantly, both metrics vary only modestly over a wide range of α values, indicating that HYBRID is robust to the precise choice of this parameter and does not require extensive tuning in practice. Although the marginal error and deletion ratio may differ in scale during the algorithm's execution, α still influences the optimization process. When the two metrics differ substantially, the algorithm tends to prioritize reducing the marginal error. As tuples are iteratively removed, the metrics eventually reach comparable scales, as observed in our experiments, at which point the effect of α becomes more pronounced. A detailed sensitivity analysis, including the behavior of the objective value and results for individual generation models is deferred to Figure 13 .

VI. RELATED WORK

We survey work on DP synthetic data generation and data repair. Our work is the first to complement and combine them via *post-hoc repair of DP synthetic data*.

Private synthetic data generation. A broad spectrum of techniques has been developed for generating DP synthetic data [3]–[10], [41]–[44]. These include methods that approximate the underlying distribution using low dimensional marginals [7], [8], [10], [11] as well as approaches based on GANs and other deep learning models [3]–[5]. Empirical studies [45] find that marginal-based systems offer some of the strongest utility in the private setting. Kamino [14] extends this line of work by generating DP synthetic data that additionally satisfies (possibly soft) DCs [12], allowing users to incorporate integrity requirements directly into the synthesis pipeline. Closest to our statistical objective, Wang et al. [15] post-process a DP release toward user-selected measures by resampling it with optimal weights. Despite these advances, generating high-quality DP synthetic data remains computationally expensive.

Data repair. Data repair restores consistency with a database by minimally modifying it [13], [21], [46]–[48], either by updating values [13], [21], [28], [47], [49], [50] or by deleting tuples [12], [17]–[19], [46], [51], [52]. We build on the latter model whose complexity is by now well understood [17]. Specifically, we derive both our baselines and Phase 1 of HYBRIDfrom ideas related to conflict graphs [20], [21]. These methods minimize change alone, and update-based systems such as HoloClean [13] preserve the statistics of the data at hand, which after DP synthesis are likely distorted. The closest formulation [19] minimizes deletions under FDs subject to hard bounds on the value proportions of a single attribute; we instead support DCs, multiple two-way targets supplied as input, and a soft relative-error objective.

VII. CONCLUSION AND FUTURE WORK

We introduced *synthetic data repair under DP*, the first framework for post-hoc repair of DP synthetic data that jointly enforces DCs and marginal targets, a complement to DP synthesizers, minimizing a weighted objective trading off deletions against relative marginal error, with $\mathcal{L}_\alpha$-optimality relative to the deletion model and the given inputs. The problem is NP-hard even with the marginal terms alone or a single DC alone. We proposed an exact MILP and a scalable two-phase greedy heuristic; on four real-world datasets and four synthesizers, repair eliminates all DC violations and reduces the targeted marginal error, often by orders of magnitude, while largely preserving global fidelity. A large deletion ratio signals a poor fit between the release, the DCs, and the targets, and can guide the choice between repair and regeneration. Future directions include richer statistical constraints and downstream fairness metrics (Section I), privately mining DCs from the private data, settling the complexity of repair with constant-size DCs, alternative marginal selection and release mechanisms, and extending the deletion-based model to update-based repairs.

AI usage disclosure. In accordance with IEEE policy, we disclose that LLMs were used during the preparation of this manuscript to review drafts for consistency and clarity and to suggest re-wordings and condensations of text written by the authors. All suggested text was reviewed, edited, and approved by the authors, who authored all technical content, proofs, experiments, and claims.

REFERENCES

[1] C. Dwork, "Differential privacy," in *Automata, Languages and Programming*, ser. Lecture Notes in Computer Science, M. Bugliesi, B. Preneel, V. Sassone, and I. Wegener, Eds. Springer Berlin Heidelberg, 2006.

[2] C. Dwork, A. Roth *et al.*, "The algorithmic foundations of differential privacy." *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3-4, pp. 211–407, 2014.

[3] R. Torkzadehmahani, P. Kairouz, and B. Paten, "DP-CGAN: differentially private synthetic data and label generation," in *CVPR*, 2019, pp. 98–104.

[4] L. Rosenblatt, X. Liu, S. Pouyanfar, E. de Leon, A. Desai, and J. Allen, "Differentially private synthetic data: Applied evaluations and enhancements," *arXiv preprint arXiv:2011.05537*, 2020.

[5] J. Yoon, J. Jordon, and M. van der Schaar, "PATE-GAN: Generating synthetic data with differential privacy guarantees," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=S1zk9iRqF7

[6] M. Hardt and G. N. Rothblum, "A multiplicative weights mechanism for privacy-preserving data analysis," *2010 IEEE 51st Annual Symposium on Foundations of Computer Science*, pp. 61–70, 2010.

[7] J. Zhang, G. Cormode, C. M. Procopiuc, D. Srivastava, and X. Xiao, "Privbayes: Private data release via bayesian networks," *ACM Transactions on Database Systems (TODS)*, vol. 42, no. 4, pp. 1–41, 2017.

[8] R. McKenna, G. Miklau, and D. Sheldon, "Winning the nist contest: A scalable and general approach to differentially private synthetic data," 2021. [Online]. Available: https://arxiv.org/abs/2108.04978

[9] Z. Huang, R. McKenna, G. Bissias, G. Miklau, M. Hay, and A. Machanavajjhala, "Psyndb: Accurate and accessible private data generation," *Proc. VLDB Endow.*, vol. 12, no. 12, pp. 1918–1921, 2019. [Online]. Available: http://www.vldb.org/pvldb/vol12/p1918-huang.pdf

[10] R. McKenna, B. Mullins, D. Sheldon, and G. Miklau, "AIM: an adaptive and iterative mechanism for differentially private synthetic data," *CoRR*, vol. abs/2201.12677, 2022. [Online]. Available: https://arxiv.org/abs/2201.12677

[11] R. McKenna, D. Sheldon, and G. Miklau, "Graphical-model based estimation and inference for differential privacy," in *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 4435–4444.

[12] J. Chomicki and J. Marcinkowski, "Minimal-change integrity maintenance using tuple deletions," *Inf. Comput.*, vol. 197, no. 1-2, pp. 90–121, 2005.

[13] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "Holoclean: Holistic data repairs with probabilistic inference," *PVLDB*, vol. 10, no. 11, pp. 1190–1201, 2017.

[14] C. Ge, S. Mohapatra, X. He, and I. F. Ilyas, "Kamino: Constraint-aware differentially private data synthesis," *Proc. VLDB Endow.*, vol. 14, no. 10, pp. 1886–1899, 2021.

[15] H. Wang, S. Sudalairaj, J. Henning, K. Greenewald, and A. Srivastava, "Post-processing private synthetic data for improving utility on selected measures," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[16] L. Bertossi, *Database Repairing and Consistent Query Answering*. Morgan & Claypool Publishers, 2011.

[17] E. Livshits, B. Kimelfeld, and S. Roy, "Computing optimal repairs for functional dependencies," *ACM Trans. Database Syst.*, vol. 45, no. 1, pp. 4:1–4:46, 2020. [Online]. Available: https://doi.org/10.1145/3360904

[18] A. Gilad, D. Deutch, and S. Roy, "On multiple semantics for declarative database repairs," in *SIGMOD*, 2020, pp. 817–831.

[19] Y. Liu, F. Shen, K. Ghosh, A. Gilad, B. Kimelfeld, and S. Roy, "The cost of representation by subset repairs," *Proc. VLDB Endow.*, vol. 18, no. 2, pp. 475–487, 2024. [Online]. Available: https://www.vldb.org/pvldb/vol18/p475-liu.pdf

[20] E. Livshits, R. Kochirgan, S. Tsur, I. F. Ilyas, B. Kimelfeld, and S. Roy, "Properties of inconsistency measures for databases," in *SIGMOD*, 2021, pp. 1182–1194.

[21] X. Chu, I. F. Ilyas, and P. Papotti, "Holistic data cleaning: Putting violations into context," in *ICDE*, 2013, pp. 458–469.

[22] S. Flood, M. King, R. Rodgers, S. Ruggles, J. R. Warren, and M. Westberry, "Integrated public use microdata series, current population survey: Version 10.0 [dataset]," Minneapolis, MN: IPUMS, 2022, 2022. [Online]. Available: https://doi.org/10.18128/D030.V10.0

[23] G. Ganev, B. Oprisanu, and E. D. Cristofaro, "Robin hood and matthew effects: Differential privacy has disparate impact on synthetic data," in *ICML*, vol. 162, 2022, pp. 6944–6959.

[24] M. Bun and T. Steinke, "Concentrated differential privacy: Simplifications, extensions, and lower bounds," in *Theory of Cryptography Conference*. Springer, 2016, pp. 635–658.

[25] M. Lichman, "Uci machine learning repository," https://archive.ics.uci.edu/ml/datasets/adult, 2013.

[26] P. Bohannon, W. Fan, F. Geerts, X. Jia, and A. Kementsietsidis, "Conditional functional dependencies for data cleaning," in *ICDE*, 2007.

[27] N. Koudas, A. Saha, D. Srivastava, and S. Venkatasubramanian, "Metric functional dependencies," in *ICDE*, 2009.

[28] M. Dallachiesa, A. Ebaid, A. Eldawy, A. K. Elmagarmid, I. F. Ilyas, M. Ouzzani, and N. Tang, "NADEEF: a commodity data cleaning system," in *SIGMOD*, 2013, pp. 541–552.

[29] X. Chu, I. F. Ilyas, and P. Papotti, "Discovering denial constraints," *Proceedings of the VLDB Endowment*, vol. 6, no. 13, pp. 1498–1509, 2013.

[30] T. Bleifuß, S. Kruse, and F. Naumann, "Efficient denial constraint discovery with hydra," *Proceedings of the VLDB Endowment*, vol. 11, no. 3, pp. 311–323, 2017.

[31] E. H. M. Pena, E. C. de Almeida, and F. Naumann, "Discovery of approximate (and exact) denial constraints," *Proc. VLDB Endow.*, vol. 13, no. 3, pp. 266–278, 2019.

[32] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of cryptography conference*. Springer, 2006, pp. 265–284.

[33] I. Kotsogiannis, Y. Tao, X. He, M. Fanaeepour, A. Machanavajjhala, M. Hay, and G. Miklau, "Privatesql: a differentially private sql query engine," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1371–1384, 2019.

[34] C. Li and G. Miklau, "An adaptive mechanism for accurate query answering under differential privacy," *Proc. VLDB Endow.*, vol. 5, no. 6, pp. 514–525, 2012. [Online]. Available: http://vldb.org/pvldb/vol5/p514_chaoli_vldb2012.pdf

[35] H. Li, L. Xiong, L. Zhang, and X. Jiang, "Dpsynthesizer: Differentially private data synthesizer for privacy preserving data sharing," *Proc. VLDB Endow.*, vol. 7, no. 13, pp. 1677–1680, 2014. [Online]. Available: http://www.vldb.org/pvldb/vol7/p1677-li.pdf

[36] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, Eds. New York: Plenum Press, 1972, pp. 85–103.

[37] ProPublica, "Compas recidivism risk score data and analysis." [Online]. Available: https://www.propublica.org/datastore/dataset/compas-recidivism-risk-score-data-and-analysis

[38] S. Mohapatra, A. Gilad, X. He, and B. Kimelfeld, "Computing inconsistency measures under differential privacy," *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 140:1–140:27, 2025. [Online]. Available: https://doi.org/10.1145/3725397

[39] V. V. Vazirani, "Approximation algorithms," *Georgia Inst. Tech*, 1997.

[40] K. L. Clarkson, "A modification of the greedy algorithm for vertex cover," *Information Processing Letters*, vol. 16, no. 1, pp. 23–25, 1983.

[41] A. Kotelnikov, D. Baranchuk, I. Rubachev, and A. Babenko, "Tabddpm: Modelling tabular data with diffusion models," in *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 17 564–17 579. [Online]. Available: https://proceedings.mlr.press/v202/kotelnikov23a.html

[42] T. Liu, J. Tang, G. Vietri, and S. Wu, "Generating private synthetic data with genetic algorithms," in *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 22 009–22 027. [Online]. Available: https://proceedings.mlr.press/v202/liu23ag.html

[43] T. Liu, G. Vietri, and S. Wu, "Iterative methods for private synthetic data: Unifying framework and new methods," in *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, 2021, pp. 690–702. [Online]. Available: https://proceedings.neurips.cc/paper/2021/hash/0678c572b0d5597d2d4a6b5bd135754c-Abstract.html

[44] G. Vietri, C. Archambeau, S. Aydöre, W. Brown, M. Kearns, A. Roth, A. A. Siva, S. Tang, and Z. S. Wu, "Private synthetic data for multitask learning and marginal queries," in *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. [Online]. Available: http://papers.nips.cc/paper_files/paper/2022/hash/7428310c0f97f1c6bb2ef1be99c1ec2a-Abstract-Conference.html

[45] Y. Tao, R. McKenna, M. Hay, A. Machanavajjhala, and G. Miklau, "Benchmarking differentially private synthetic data generation algorithms," *CoRR*, vol. abs/2112.09238, 2021.

[46] F. N. Afrati and P. G. Kolaitis, "Repair checking in inconsistent databases: Algorithms and complexity," in *ICDT*, 2009, pp. 31–41.

[47] L. E. Bertossi, S. Kolahi, and L. V. S. Lakshmanan, "Data cleaning and query answering with matching dependencies and matching functions," *Theory Comput. Syst.*, vol. 52, no. 3, pp. 441–482, 2013.

[48] R. Fagin, B. Kimelfeld, and P. G. Kolaitis, "Dichotomies in the complexity of preferred repairs," in *PODS*, 2015, pp. 3–15.

[49] F. Geerts, G. Mecca, P. Papotti, and D. Santoro, "The LLUNATIC data-cleaning framework," *Proc. VLDB Endow.*, vol. 6, no. 9, pp. 625–636, 2013.

[50] S. Kolahi and L. V. S. Lakshmanan, "On approximating optimum repairs for functional dependency violations," in *Proceedings of the 12th International Conference on Database Theory*, ser. ICDT '09. New York, NY, USA: Association for Computing Machinery, 2009, p. 53–62. [Online]. Available: https://doi.org/10.1145/1514894.1514901

[51] A. Lopatenko and L. E. Bertossi, "Complexity of consistent query answering in databases under cardinality-based and incremental repair semantics," in *ICDT*, 2007, pp. 179–193.

[52] D. Miao, Z. Cai, J. Li, X. Gao, and X. Liu, "The computation of optimal subset repairs," *Proc. VLDB Endow.*, vol. 13, no. 12, p. 2061–2074, jul 2020.

[53] I. Mironov, "Rényi differential privacy," in *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*. IEEE, 2017, pp. 263–275.

[54] F. D. McSherry, "Privacy integrated queries: an extensible platform for privacy-preserving data analysis," in *SIGMOD*, 2009, pp. 19–30.

[55] F. McSherry and K. Talwar, "Mechanism design via differential privacy," in *Proceedings of the 48th Annual IEEE Symposium on Foundations of Computer Science*, ser. FOCS '07. USA: IEEE Computer Society, 2007, p. 94–103. [Online]. Available: https://doi.org/10.1109/FOCS.2007.41

[56] D. Durfee and R. M. Rogers, "Practical differentially private top-k selection with pay-what-you-get composition," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, 2019, pp. 3527–3537. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/b139e104214a08ae3f2ebcce149cdf6e-Abstract.html

[57] G. Qiao, W. J. Su, and L. Zhang, "Oneshot differentially private top-k selection," *arXiv preprint arXiv:2105.08233*, 2021.

[58] J. Dong, D. Durfee, and R. Rogers, "Optimal differential privacy composition for exponential mechanisms," in *International Conference on Machine Learning*. PMLR, 2020, pp. 2597–2606.

[59] M. Cesar and R. Rogers, "Bounding, concentrating, and truncating: Unifying privacy loss composition for data analytics," in *Algorithmic Learning Theory*. PMLR, 2021, pp. 421–457.

[60] R. Rogers and T. Steinke, "A better privacy analysis of the exponential mechanism," DifferentialPrivacy.org, 07 2021, https://differentialprivacy.org/exponential-mechanism-bounded-range/.

APPENDIX

The repair problem in Definition 8 treats the monitored marginals $\mathcal{M}$ and their noisy target values as inputs. Our repair algorithms are agnostic to how these targets are selected or released, and this section is an optional component of the framework. For completeness, this section describes one possible DP procedure for obtaining such targets when the data owner retains access to the private dataset. The procedure prioritizes two-way marginals that differ substantially between the private and synthetic datasets, using one-shot top-k selection followed by Gaussian measurement.

A. Differential Privacy Background

We complete the background of Section II-B with the definitions used only by the mechanism below and by its analysis in Section F.

Definition 14 (Zero-Concentrated Differential Privacy (zCDP) [24])**.** *A mechanism $\mathcal{A}$ is said to be* ρ-zero-concentrated differential private*, or ρ-zCDP for short, if for any neighboring datasets D and D' and all $\alpha \in (1, \infty)$ it holds that*

$$D_\alpha(\mathcal{A}(D) \| \mathcal{A}(D')) \leq \rho\alpha$$

where $D_\alpha(\mathcal{A}(D)\|\mathcal{A}(D'))$ denotes the Rényi divergence of the distribution $\mathcal{A}(D)$ from the distribution $\mathcal{A}(D')$ at order α [53].

Intuitively, zCDP bounds the divergence between the output distributions on neighboring databases, so that no single tuple can influence the output too much. zCDP composes additively across sequential private releases:

Proposition 15 (Sequential composition for zCDP [24], [54])**.** *Let $\mathcal{A}_1 : \mathcal{D} \to \mathcal{Y}$ and $\mathcal{A}_2 : \mathcal{Y} \to \mathcal{Z}$. Suppose $\mathcal{A}_1$ satisfies ρ_1-zCDP, $\mathcal{A}_2(\cdot)$ satisfies ρ_2-zCDP. Define $\mathcal{A}_3 : \mathcal{D} \to \mathcal{Z}$ by $\mathcal{A}_3(D) = \mathcal{A}_2(\mathcal{A}_1(D))$. Then, $\mathcal{A}_3$ satisfies $(\rho_1 + \rho_2)$-zCDP.*

For a scalar function ($k = 1$) we write the ℓ_2 sensitivity of Definition 4 simply as Δ_f (e.g., Δ_u for a utility function u).

a) The Exponential Mechanism (EM): Given a set of candidates $\mathcal{R}$ and a utility function $u : \mathcal{R} \times \mathcal{D} \to \mathbb{R}$ with sensitivity Δ_u, the EM [55] selects a candidate $r \in \mathcal{R}$ with probability proportional to $\exp\big(\epsilon\, u(D, r)/(2\Delta_u)\big)$, thus favoring high-utility candidates. It satisfies ϵ-DP, and hence $\frac{1}{2}\epsilon^2$-zCDP [24].

B. The Mechanism

The algorithm (pseudocode in Section G) gets two privacy parameters, one for top-k selection, ρ_{Topk}, and one for noisily estimating them, ρ_{marg}. It proceeds in two stages. In the *selection* stage, it computes all candidate 2-way marginals of D_p and D_s and scores each candidate by the utility function u (Definition 17), perturbed with Gumbel noise of scale $2\Delta_u\sqrt{k/(8\rho_{Topk})}$, where Δ_u is the sensitivity of u; the k candidates with the highest noisy scores are selected. This one-shot top-k selection [56], [57] is equivalent to k sequential Exponential Mechanisms, satisfies ρ_{Topk}-zCDP [56], [58]–[60], and avoids a linear privacy blowup in the number of candidates. In the *measurement* stage, each selected marginal is released as the vector f of all its entries via the Gaussian Mechanism with budget ρ_{marg}/k; replacing one tuple of D_p changes at most two entries by $1/n$, where $n = |D_p|$, so $\Delta_2 f = \sqrt{2}/n$ and $\sigma_{\text{marg}} = \sqrt{k}/(n\sqrt{\rho_{marg}})$. The noisy entries are then clipped to $[0, 1]$ and renormalized. The output $\mathcal{M}$ consists of the value-pair entries of the selected marginals with their noisy estimates. We thus get the following result, slightly abusing notation to denote by $\mathcal{M}_0$ the set of all marginals (proof in Section F).

Proposition 16 (Guarantees for the illustrative procedure)**.** *Given the utility function u with sensitivity Δ_u, a set of marginals $\mathcal{M}_0$, privacy parameters $(\rho_{Topk}, \rho_{marg})$, and a natural number k, the following holds:*

1) *The described procedure satisfies $(\rho_{marg} + \rho_{Topk})$-zCDP.*
2) *Denote by $OPT^{(i)}$ the i-th highest (true) score, and by $\mathcal{M}_0^{(i)}$ the i-th noisy private marginal selected by the one-shot top-k mechanism. For $\forall t$ and $\forall i \in \{1, 2, \ldots, k\}$, we have*
$$Pr[u(\mathcal{M}_0^{(i)}) \leq OPT^{(i)} - \frac{2\Delta_u}{\sqrt{8\rho_{Topk}/k}}(\ln(|\mathcal{M}_0|) + t)] \leq e^{-t}$$

C. Utility Function and Attribute Focus

Our utility function prioritizes the marginals most poorly represented in the synthetic data; The above procedure accepts any utility function.

Definition 17 (2-way marginals utility)**.** *Given a private database D_p, a synthetic database D_s over $\mathcal{A}ttr$, and two attributes $A_i, A_j \in \mathcal{A}ttr$, the utility of the marginal on (A_i, A_j) is the ℓ_1 distance between its tables in the two databases:*

$$u(A_i, A_j) = \sum_{(a_i, a_j)} \big|m_{D_p}(a_i, a_j) - m_{D_s}(a_i, a_j)\big|,$$

where the sum ranges over $(a_i, a_j) \in Dom(A_i) \times Dom(A_j)$.

The sensitivity of the function is low, allowing us to compute it with relative accuracy under DP (proof in Section F).

Proposition 18. *The sensitivity of u is $\Delta_u = \frac{2}{n}$.*

When users prefer a specific subset of attributes, the algorithm can restrict its candidates to that subset (as in Example 1).

This is only one acquisition strategy; applications may instead use domain-specified, fairness-relevant, or workload-relevant marginals, or measurements already produced during synthesis.

D. Proof of Item (1) of Proposition 9

Proof. We reduce from the MAX-CUT decision problem, which is NP-complete [36]. Given an undirected graph $G = (V, E)$ and an integer k with $1 \leq k \leq |E|$, MAX-CUT asks whether there exists $S \subseteq V$ such that $\text{cut}(S) = |\{(u, v) \in E : |S \cap \{u, v\}| = 1\}| \geq k$. Throughout, per Definition 3, the marginal of a non-empty repaired dataset D_r is normalized by $|D_r|$, and the marginals of the empty repair

are 0 by convention, so each of its relative errors equals 1 and $\mathcal{L}(\emptyset) = 1$.

Construction. Let $n = |V|$ and $N = 4|E|\,n$. We create a relation D_s with schema $\mathcal{A}ttr = \{A_e : e \in E\} \cup \{X, B\}$, where each A_e and X are binary and $Dom(B) = \{0, 1, \ldots, N\}$, containing: (i) a *vertex tuple* t_v for each $v \in V$, with $t_v[A_e] = 1$ iff $v \in e$, $t_v[X] = 1$, and $t_v[B] = 0$; and (ii) N *ballast tuples* $u_1, \ldots, u_N$, with $u_i[A_e] = 0$ for all $e \in E$, $u_i[X] = 1$, and $u_i[B] = i$ (the attribute B only serves to make the ballast tuples pairwise distinct; it is not monitored). Thus $|D_s| = N + n$. We set $\Sigma = \emptyset$ (no DCs), $\alpha = 0$, and for each edge $e \in E$ define a marginal monitoring $(A_e{=}1, X{=}1)$ with target $\tilde{m}(A_e{=}1, X{=}1) = 1/(N{+}n)$; thus $|\mathcal{M}| = |E|$. Since every target is positive, the relative marginal errors in $\mathcal{L}$ are well-defined. The decision threshold for $\mathcal{L}(D_r)$ is $\lambda = (|E| - k + \frac{1}{2})/|E|$. The instance has $N{+}n = O(|E|\,n)$ tuples, $|E| + 2$ attributes, and $|E|$ marginals, so the reduction is polynomial.

The objective as a function of the repair. Consider first any non-empty repair $D_r \subseteq D_s$ retaining the vertex set $S = \{v : t_v \in D_r\}$ (with $s = |S|$) and $b \in [0, N]$ ballast tuples, so $|D_r| = b + s \geq 1$. For each edge e, only vertex tuples satisfy $A_e = 1$, hence the retained count for the monitored pair $(A_e{=}1, X{=}1)$ is $\kappa_e = |S \cap e| \in \{0, 1, 2\}$ and $m_{D_r}(A_e{=}1, X{=}1) = \kappa_e/(b{+}s)$. Writing $f = (N{+}n)/(b{+}s)$ and noting $b + s \leq N + n$, we have $f \geq 1$ and, with $\alpha = 0$,

$$\mathcal{L}(D_r) \;=\; \frac{1}{|E|}\sum_{e\in E}\frac{\big|\kappa_e/(b+s) - 1/(N+n)\big|}{1/(N+n)} \;=\; \frac{1}{|E|}\sum_{e\in E}\big|\kappa_e f - 1\big|. \tag{5}$$

Since $f \geq 1$, each term equals 1 if $\kappa_e = 0$, equals $f - 1 \geq 0$ if $\kappa_e = 1$, and equals $2f - 1 \geq 1$ if $\kappa_e = 2$.

Lower bound (holds for every repair). From the case analysis above, every edge with $\kappa_e \in \{0, 2\}$ contributes at least 1 to the sum in (5), and edges with $\kappa_e = 1$ are exactly the edges cut by S. For the empty repair, every relative error equals 1 by convention, matching the $\kappa_e = 0$ case with $S = \emptyset$ and $\mathrm{cut}(\emptyset) = 0$. Hence, for *every* repair $D_r \subseteq D_s$,

$$|E| \cdot \mathcal{L}(D_r) \;\geq\; |E| - \mathrm{cut}(S). \tag{6}$$

($\Rightarrow$) Suppose $\mathrm{cut}(S) \geq k$ for some $S \subseteq V$, and take $D_r = \{t_v : v \in S\} \cup \{u_1, \ldots, u_N\}$, i.e., retain S and *all* ballast tuples ($b = N$; note $D_r \neq \emptyset$ since $N \geq 1$). Then $f = (N + n)/(N + s)$, so $0 \leq f - 1 = (n - s)/(N + s) \leq n/N = 1/(4|E|)$. Splitting the sum in (5) by the value of κ_e and using $2f - 1 = 2(f - 1) + 1$ to split each doubly-covered edge's term, so that the trailing $+1$ (one per edge with $\kappa_e{=}2$) joins the $\kappa_e{=}0$ edges in the constant, while $\mathrm{cut}(S) = \#\{\kappa_e{=}1\}$ gives $\#\{\kappa_e{=}0\} + \#\{\kappa_e{=}2\} = |E| - \mathrm{cut}(S)$, we obtain

$$\begin{aligned}
|E| \cdot \mathcal{L}(D_r) &= \#\{\kappa_e{=}0\} + \#\{\kappa_e{=}1\}\,(f-1) + \#\{\kappa_e{=}2\}\,(2f-1) \\
&= \underbrace{\big(\#\{\kappa_e{=}0\} + \#\{\kappa_e{=}2\}\big)}_{=\,|E|-\mathrm{cut}(S)} + \big(\#\{\kappa_e{=}1\} + 2\,\#\{\kappa_e{=}2\}\big)(f-1) \\
&\leq \big(|E| - \mathrm{cut}(S)\big) + 2|E| \cdot \frac{1}{4|E|} \;\leq\; |E| - k + \tfrac{1}{2},
\end{aligned}$$

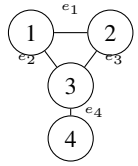
hence $\mathcal{L}(D_r) \leq \lambda$.

($\Leftarrow$) Suppose $\mathcal{L}(D_r^*) \leq \lambda$ for some $D_r^* \subseteq D_s$, and let $S^* = \{v : t_v \in D_r^*\}$. By the lower bound (6), $|E| - \mathrm{cut}(S^*) \leq |E| \cdot \mathcal{L}(D_r^*) \leq |E| \cdot \lambda = |E| - k + \frac{1}{2}$, so $\mathrm{cut}(S^*) \geq k - \frac{1}{2}$, and since $\mathrm{cut}(S^*)$ is an integer, $\mathrm{cut}(S^*) \geq k$.

Since MAX-CUT is NP-complete and the construction uses $\Sigma = \emptyset$, the decision version of Definition 8 is NP-hard even without DCs; the hardness arises entirely from the marginal terms of the objective. Membership in NP is immediate (a consistent repair D_r is a polynomial certificate and $\mathcal{L}(D_r)$ is computable in polynomial time), so the decision version is in fact NP-complete. □

Intuitively, the lower bound (6) holds for *every* repair, regardless of how many ballast tuples it retains; the ballast is needed only in the ($\Rightarrow$) direction, where retaining all of it pins the normalizing denominator $|D_r|$ close to $N + n$, so that edges with exactly one retained endpoint contribute a negligible relative error ($f - 1 \leq 1/(4|E|)$). Without the ballast, the denominator $|D_r| = |S|$ varies with the chosen cut and the objective is no longer a monotone function of $\mathrm{cut}(S)$.

Example 19. We give the full version of Example 10. Consider the graph G with vertices $\{v_1, v_2, v_3, v_4\}$ and edges $e_1 = (v_1, v_2)$, $e_2 = (v_1, v_3)$, $e_3 = (v_2, v_3)$, $e_4 = (v_3, v_4)$.



Here $n = 4$ and $|E| = 4$, so the construction sets $N = 4|E|n = 64$, $\alpha = 0$, and targets $\tilde{m}(A_e{=}1, X{=}1) = 1/68$, producing the relation in Table II (68 tuples in total). The maximum cut of G has value 3, achieved e.g. by $S = \{v_1, v_2, v_4\}$, which cuts e_2, e_3, e_4. Retaining S together with all 64 ballast tuples gives $|D_r| = 67$ and $f = 68/67$, so the four monitored relative errors are $|2f - 1| = 69/67$ (for e_1, with both endpoints retained) and $f - 1 = 1/67$ (for each of e_2, e_3, e_4), yielding $\mathcal{L}(D_r) = \frac{1}{4}\big(\frac{69}{67} + \frac{3}{67}\big) = \frac{18}{67} \approx 0.269$. For $k = 3$ the threshold is $\lambda = (4 - 3 + \frac{1}{2})/4 = 0.375 \geq \mathcal{L}(D_r)$, certifying a cut of size at least 3; for $k = 4$ we have $\lambda = 0.125$, and indeed an exhaustive check confirms that no repair attains $\mathcal{L} \leq 0.125$, matching the fact that G has no cut of size 4. In contrast, dropping the ballast breaks the encoding. Retaining only $S = \{v_1, v_3\}$ (also a maximum cut, cutting e_1, e_3, e_4 while e_2 is doubly covered) gives $|D_r| = 2$, so $f = (N + n)/|D_r| = 68/2 = 34$ and the four monitored relative errors are $|2f - 1| = 67$ for e_2 ($\kappa = 2$) and $|f - 1| = 33$ for each of e_1, e_3, e_4 ($\kappa = 1$), yielding $\mathcal{L}(D_r) = \frac{1}{4}(33 + 67 + 33 + 33) = 41.5$. This is *worse* than retaining all four vertex tuples without ballast ($|D_r| = 4$, $f = 17$, every edge doubly covered, $\mathcal{L} = \frac{1}{4}(4 \cdot 33) = 33$), and worse still than the empty repair ($\mathcal{L}(\emptyset) = 1$). Hence, without ballast, minimizing $\mathcal{L}$ would return the empty repair—encoding a cut of 0—rather than the maximum cut of value 3, so the objective no longer tracks the cut. (The value $\mathcal{L} = 1$

for retaining all four vertex tuples is attained only when the 64 ballast tuples are *also* kept, giving $f = 1$; that is the with-ballast regime used above, not a no-ballast repair.)

E. Proof of Item (2) of Proposition 9

Proof. We reduce from the INDEPENDENT SET decision problem, which is NP-complete [36]: given an undirected graph $G = (V, E)$ and an integer K with $1 \le K \le |V|$, decide whether G has an independent set of size at least K. Let $n = |V|$ and let $\bar{E} = \{\{u, v\} : u, v \in V,\ u \neq v,\ \{u, v\} \notin E\}$ denote the set of non-edges of G.

Construction. We create a relation D_s with schema $\mathcal{A}ttr = \{P, Q, A_0\} \cup \{A_f : f \in \bar{E}\}$ containing: (i) a *vertex tuple* t_v for each $v \in V$, with $t_v[P] = 1$, $t_v[Q] = 1$, $t_v[A_0] = v$, and, for each non-edge $f \in \bar{E}$, $t_v[A_f] = f$ if $v \in f$ and $t_v[A_f] = \perp_v$ otherwise, where $\perp_v$ is a value unique to v and distinct from all non-edge labels; and (ii) $N = n$ *ballast tuples* $u_1, \ldots, u_N$ with $u_i[P] = 0$, $u_i[Q] = 1$, and a fresh constant on all remaining attributes. Thus $|D_s| = N + n = 2n$. We set $\alpha = 1/2$ and define the single DC

$$\sigma : \forall t_1, t_2\ \neg\Big(t_1[P] = 1 \wedge t_2[P] = 1 \wedge t_1[A_0] \neq t_2[A_0] \wedge \bigwedge_{f \in \bar{E}} t_1[A_f] = t_2[A_f]\Big),$$

which uses only predicates of the forms $t_i[A_j] \circ v$ and $t_1[A_j] \circ t_2[A_j]$ permitted by our DC class and has $|\bar{E}| + 3 = O(n^2)$ predicates. Finally, we define a single marginal monitoring the value pair $(P{=}1, Q{=}1)$ with target $\tilde{m}(P{=}1, Q{=}1) = K/(N + K) > 0$, and the decision threshold $\lambda = \alpha\,(n - K)/(N + n)$. The construction is polynomial in $|G|$.

The conflict graph of σ is exactly G. Ballast tuples never participate in a violation, as they fail the predicate $t[P] = 1$. Consider two distinct vertex tuples t_u, t_v. They always disagree on A_0 (its values are unique per vertex), and they agree on A_f iff $t_u[A_f] = t_v[A_f] = f$, which holds iff $f = \{u, v\} \in \bar{E}$. Hence t_u, t_v jointly violate σ iff they disagree on *every* attribute in the conjunction, i.e., iff $\{u, v\} \notin \bar{E}$, i.e., iff $\{u, v\} \in E$. (No tuple conflicts with itself, since every tuple agrees with itself on A_0.) Consequently, a repair $D_r \subseteq D_s$ is consistent iff its retained vertex set $S = \{v : t_v \in D_r\}$ is an independent set of G, while the number $b \in [0, N]$ of retained ballast tuples is unconstrained.

The objective. All tuples satisfy $Q = 1$ and exactly the vertex tuples satisfy $P = 1$, so a non-empty repair D_r keeping S (with $s = |S|$) and b ballast tuples has $m_{D_r}(P{=}1, Q{=}1) = s/(b + s)$ and

$$\mathcal{L}(D_r) = \alpha \cdot \frac{(N + n) - (b + s)}{N + n} + (1{-}\alpha) \cdot \frac{\big|\, s/(b + s) - \frac{K}{N+K} \,\big|}{K/(N + K)}.$$

($\Rightarrow$) Suppose G has an independent set of size at least K. Retain exactly K of its vertices (any subset of an independent set is independent) together with all N ballast tuples. Then $s/(b + s) = K/(N + K)$, so the marginal term vanishes, and $\mathcal{L}(D_r) = \alpha\,(n - K)/(N + n) = \lambda$.

($\Leftarrow$) Suppose every independent set of G has size at most $K - 1$, and consider any consistent non-empty repair, with $s \le K - 1$ retained vertex tuples and $b \le N$ retained ballast tuples. Then $b{+}s \le N{+}K{-}1$, so the deletion term alone is at least $\alpha\,\big((N{+}n){-}(N{+}K{-}1)\big)/(N{+}n) = \lambda{+}\alpha/(N{+}n) > \lambda$, and since the marginal term is non-negative, $\mathcal{L}(D_r) > \lambda$. The empty repair has $\mathcal{L}(\emptyset) = 1 > \lambda$ as well. Hence no consistent repair attains $\mathcal{L} \le \lambda$.

Since INDEPENDENT SET is NP-complete and the construction is polynomial, the decision version of Definition 8 is NP-hard (indeed NP-complete, with membership as in the proof of Item (1)) already for $|\Sigma| = |\mathcal{M}| = 1$. The argument applies verbatim for every fixed $\alpha \in (0, 1]$. □

F. Proof of Item (3) of Proposition 9

Proof. We reuse the vertex tuples and the single DC σ from the proof of Item (2), omitting the ballast tuples, and set $\mathcal{M} = \emptyset$, $\alpha = 1$, and threshold $\lambda = (n - K)/n$. By the convention of Definition 6, the marginal term vanishes when $\mathcal{M} = \emptyset$, so $\mathcal{L}(D_r) = (n - |D_r|)/n$ and $\mathcal{L}(D_r) \le \lambda$ iff $|D_r| \ge K$. As shown in the proof of Item (2), a repair is consistent iff its retained vertex set is an independent set of G; hence a consistent repair of size at least K exists iff G has an independent set of size at least K (any subset of an independent set is independent). The empty repair has $\mathcal{L}(\emptyset) = \alpha = 1 > \lambda$ since $K \ge 1$. NP-hardness (and NP-completeness) follows as before. Note that α is part of the input and the instance above fixes $\alpha = 1$; any fixed $\alpha > 0$ works by scaling λ accordingly, whereas for $\alpha = 0$ and $\mathcal{M} = \emptyset$ the objective is identically 0 and the problem is trivial. □

Proof. Let D_s be a (non-empty) synthetic database with $n = |D_s| \ge 1$, Σ a set of DCs, $\mathcal{M}$ a set of marginals, and $\alpha \in [0, 1]$. The MILP variables are $x_l \in \{0, 1\}$ for $t_l \in D_s$, $d_m \ge 0$ for $m \in \mathcal{M}$, and $z_{ml} \ge 0$ for $m \in \mathcal{M}$, $l \in [1, n]$. For a feasible point (x, d, z) we write $D_r(x) = \{t_l \mid x_l = 1\}$ and, for $m = (A_i{=}a_1, A_j{=}a_2)$, $C_m(x) = \sum_{l:\, t_l[A_i]=a_1,\, t_l[A_j]=a_2} x_l$. We take as a *standing precondition on the MILP input* that each target satisfies $\tilde{m}(a_1, a_2) \in (0, 1]$. This is an assumption on the targets alone, *independent of how they were produced*: positivity is required by Definition 5, and the upper bound $\tilde{m}(a_1, a_2) \le 1$ holds for any target after the source-independent preprocessing that clips each estimate to $[0, 1]$ (the remark following this proof shows this clipping never increases the target's error, and gives a variant of the program that dispenses with it entirely and admits arbitrary $\tilde{m}(a_1, a_2) > 0$). In particular the DP mechanism of Algorithm 6 already returns estimates in $[0, 1]$, but no step of this proof relies on that mechanism. Recall also that, by the convention of Definition 3, the marginals of the empty repair are 0, so each of its relative errors equals 1 and $\mathcal{L}(\emptyset) = 1$.

Step 1: exactness of the linearization. At every feasible point, $z_{ml} = d_m \cdot x_l$: if $x_l = 1$, then $z_{ml} \le d_m$ and $z_{ml} \ge d_m - (1 - x_l) = d_m$ give $z_{ml} = d_m$; if $x_l = 0$, then $z_{ml} \le x_l = 0$ and $z_{ml} \ge 0$ give $z_{ml} = 0$. Consequently $\sum_l z_{ml} = d_m \sum_l x_l$ at every feasible point. (When $x_l = 0$, the constraint $z_{ml} \ge d_m - 1$ additionally forces $d_m \le 1$; by Step 3 this never excludes an intended solution, since the repaired marginal lies in $[0, 1]$

and, by the input precondition, so does each target $\tilde{m}(a_1, a_2)$, so every required distance $|m_{D_r}(A_i{=}a_1, A_j{=}a_2) - \tilde{m}(a_1, a_2)|$ is at most 1.)

Step 2: every feasible point over-approximates $\mathcal{L}$. Let (x, d, z) be feasible and $D_r = D_r(x)$. First, $D_r \models \Sigma$: if a pair $(t_i, t_j) \in D_r$ violated some $\sigma \in \Sigma$, then $x_i = x_j = 1$ would give $x_i + x_j = 2 > 1$, contradicting feasibility. Second, we claim $d_m / \tilde{m}(a_1, a_2) \geq E_m(m_{D_r}(A_i{=}a_1, A_j{=}a_2))$ for every m. If $\sum_l x_l \geq 1$, by Step 1 the two distance constraints read $d_m \sum_l x_l \geq \pm\big(C_m(x) - \tilde{m}(a_1, a_2) \sum_l x_l\big)$; dividing by $\sum_l x_l \geq 1 > 0$ yields $d_m \geq |\, C_m(x)/\sum_l x_l - \tilde{m}(a_1, a_2)\,| = |m_{D_r}(A_i{=}a_1, A_j{=}a_2) - \tilde{m}(a_1, a_2)|$, proving the claim. If instead $\sum_l x_l = 0$, i.e., $D_r = \emptyset$, the safeguard constraint gives $d_m \geq \tilde{m}(a_1, a_2)$, so $d_m / \tilde{m}(a_1, a_2) \geq 1 = E_m(m_\emptyset(A_i{=}a_1, A_j{=}a_2))$, proving the claim as well. Since also $(n - \sum_l x_l)/n = (n - |D_r|)/n$, the MILP objective at (x, d, z) is at least $\mathcal{L}(D_r)$ in both cases.

Step 3: every repair is realized with objective exactly $\mathcal{L}$. Conversely, let D_r be any DC-consistent subset of D_s, possibly empty. Set $x_l = \mathbb{1}[t_l \in D_r]$, $d_m = |m_{D_r}(A_i{=}a_1, A_j{=}a_2) - \tilde{m}(a_1, a_2)|$, and $z_{ml} = d_m x_l$ for each $m \in \mathcal{M}$, $l \in [1, n]$ (for $D_r = \emptyset$ this reads $x = 0$, $d_m = \tilde{m}(a_1, a_2)$, $z = 0$). Then $d_m \in [0, 1]$, since the repaired marginal lies in $[0, 1]$ and, by the input precondition, so does the target $\tilde{m}(a_1, a_2)$. All constraints hold: the DC constraints because D_r is consistent; the two distance constraints with equality, since $\sum_l z_{ml} = d_m \sum_l x_l = |C_m(x) - \tilde{m}(a_1, a_2) \sum_l x_l|$ (both sides are 0 when $D_r = \emptyset$); the safeguard constraints, with equality when $D_r = \emptyset$ and with non-positive right-hand side otherwise; and the linearization constraints directly (for $x_l = 0$, $z_{ml} = 0 \geq d_m - 1$ uses $d_m \leq 1$). The MILP objective at this point equals $\mathcal{L}(D_r)$ exactly (in particular, $\alpha + (1 - \alpha) = 1 = \mathcal{L}(\emptyset)$ when $D_r = \emptyset$).

Step 4: conclusion. Step 3 applied to $D_r = \emptyset$ shows that the MILP is always feasible. Let (x^*, d^*, z^*) be an optimal solution with objective value OPT, and let $D_r^* = D_r(x^*)$. For every DC-consistent $D_r \subseteq D_s$,

$$\mathcal{L}(D_r^*) \overset{\text{Step 2}}{\leq} \text{OPT} \overset{\text{Step 3}}{\leq} \mathcal{L}(D_r).$$

Hence D_r^* minimizes $\mathcal{L}$ over all DC-consistent subsets of D_s, i.e., it is an $\mathcal{L}_\alpha$-optimal synthetic data repair under Definition 6. Moreover, taking $D_r = D_r^*$ shows $\text{OPT} = \mathcal{L}(D_r^*)$. Note that the argument covers all $\alpha \in [0, 1]$: no step relies on d_m having a strictly positive coefficient in the objective. □

Remark (agnostic to the target source). The proof of Proposition 11 uses only one property of the private estimates, namely the input precondition $\tilde{m}(a_1, a_2) \in (0, 1]$; it never refers to Algorithm 6 or to any particular releasing mechanism. Correctness is therefore independent of the target source, consistent with the framing in Section A: the targets may be analyst-specified, reused from statistics the synthesizer already released, or produced by the DP procedure of Algorithm 6. Two observations make the precondition harmless for an arbitrary source.

(i) Clipping to $[0, 1]$ is lossless. Any target can be forced to satisfy the precondition by the deterministic preprocessing $\tilde{m}(a_1, a_2) \mapsto \min\{1, \tilde{m}(a_1, a_2)\}$ (values are already positive). Since every attainable marginal of D_p lies in $[0, 1]$, this map is the Euclidean projection onto the interval that contains the ground-truth value $m_{D_p}(A_i{=}a_1, A_j{=}a_2)$. Projection onto a convex set is non-expansive, so $|\min\{1, \tilde{m}(a_1, a_2)\} - m_{D_p}(A_i{=}a_1, A_j{=}a_2)| \leq |\tilde{m}(a_1, a_2) - m_{D_p}(A_i{=}a_1, A_j{=}a_2)|$: clipping never increases the target's error with respect to the true marginal, and therefore only weakens the noise a downstream repair must accommodate. Because clipping depends on neither the estimate's provenance nor its noise model, it is a generic, source-independent front-end; Algorithm 6 already applies it internally (Line 14), but an external source is handled identically.

(ii) Targets outside $[0, 1]$ need no clipping. If one prefers to consume a raw estimate $\tilde{m}(a_1, a_2) > 1$ verbatim, the program of Equation (4) extends without any change to its optimum. Fix, for each m, the a-priori bound $U_m := \max\{1, \tilde{m}(a_1, a_2)\}$, and replace the linearization constraints $z_{ml} \leq x_l$ and $z_{ml} \geq d_m - (1 - x_l)$ by

$$z_{ml} \leq U_m\, x_l, \qquad z_{ml} \geq d_m - U_m(1 - x_l),$$

keeping $z_{ml} \leq d_m$ and $z_{ml} \geq 0$. This is the exact McCormick envelope of $z_{ml} = d_m x_l$ over $d_m \in [0, U_m]$: for binary x_l it still forces $z_{ml} = d_m$ when $x_l = 1$ and $z_{ml} = 0$ (with $d_m \leq U_m$) when $x_l = 0$. The bound U_m is valid because the required distance never exceeds it: $|m_{D_r}(A_i{=}a_1, A_j{=}a_2) - \tilde{m}(a_1, a_2)| \leq \max\{\tilde{m}(a_1, a_2), 1 - 0\} \leq U_m$ for every non-empty repair, and the empty-repair distance equals $\tilde{m}(a_1, a_2) \leq U_m$. Hence the safeguard $d_m \geq \tilde{m}(a_1, a_2)(1 - \sum_l x_l)$ can no longer conflict with the linearization, so the MILP remains feasible (the empty repair is realized as in Step 3) and exact (Steps 1–3 go through verbatim with 1 replaced by U_m) for any $\tilde{m}(a_1, a_2) > 0$. When $\tilde{m}(a_1, a_2) \leq 1$ we have $U_m = 1$ and the formulation reduces to Equation (4).

Algorithm 2: State Initialization

Input : Dataset D, Marginal set $\mathcal{M}$ with targets $\{\tilde{m}(a_i, a_j)\}$, parameters $\alpha \in [0, 1]$
Output: Counts c, Indicator map $\mathcal{I}$, current size n_{cur}, original size $n = |D|$

```
// Materialize counts for each marginal
for m ∈ M over (A_i=a_i, A_j=a_j) do
    c_m ← |{t ∈ D | t[A_i] = a_i ∧ t[A_j] = a_j}|;
// Pre-compute tuple-constraint indicator
for t ∈ D do
    for m ∈ M do
        I(t, m) ← 𝟙[t[A_i] = a_i ∧ t[A_j] = a_j];
n_cur ← |D|;
return (c, I, n_cur);
```

Incremental Maintenance Strategy. A naive recalculation of the loss function in a hypothetical removal of a tuple at each iteration necessitates a full dataset scan, incurring a prohibitive $\mathcal{O}(n \cdot |\mathcal{M}|)$ computational cost that severely limits scalability.

Algorithm 3: Loss Calculation on Hypothetical Removal

Input : Set of k tuples $\{t_1, \ldots, t_k\}$, Counts c, Indicator map $\mathcal{I}$, current size n_{cur}, targets $\{\tilde{m}(a_i, a_j)\}$, parameters α, $|\mathcal{M}|$, n

Output: Loss $\mathcal{L}(D_r \setminus \{t_1, \ldots, t_k\})$

$n_{new} \leftarrow n_{cur} - k$;

$m_{new} \leftarrow \frac{c_m - \sum_{i=1}^{k} \mathcal{I}(t_i, m)}{n_{new}}$;

$E_m \leftarrow \frac{|m_{new} - \tilde{m}(a_i, a_j)|}{\tilde{m}(a_i, a_j)}$;

return $\alpha \cdot \frac{n - n_{new}}{n} + (1 - \alpha) \cdot \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} E_m$;

Algorithm 4: Incremental State Update

Input : Removed tuples $t_1^*, \ldots, t_k^*$, counts c, indicator map $\mathcal{I}$, current size n_{cur}

Output: Updated counts c, updated size n_{cur}

// Decrement counts for affected marginals

for $m \in \mathcal{M}$ **do**

  $c_m \leftarrow c_m - \sum_{i=1}^{k} \mathcal{I}(t_i^*, m)$

$n_{\text{cur}} \leftarrow n_{\text{cur}} - k$;

return $(c,\ n_{cur})$;

To mitigate this, we propose an incremental maintenance strategy that decouples the loss calculation from the raw data by materializing the satisfaction states of the constraints. Our strategy follows a three-phase lifecycle: initialization, calculation, and state update, as detailed in Algorithms 2 to 4.

State Initialization (Algorithm 2). To avoid expensive data access during iterative refinement, we materialize the initial state of the dataset. We compute a frequency vector c, where each entry c_m stores the absolute count of tuples related to a marginal $m \in \mathcal{M}$ (i.e., $c_m = |\{t \in D \mid t[A_i] = a_i \wedge t[A_j] = a_j\}|$ for the marginal over $(A_i = a_i, A_j = a_j)$). In practice, this is efficiently computed via indexed `GROUP BY` operations. We also store the targets $\tilde{m}(a_i, a_j)$ for each $m \in \mathcal{M}$ and the current dataset size $n_{\text{cur}} = |D_s|$. Furthermore, we pre-calculate an indicator matrix $\mathcal{I} : D \times \mathcal{M} \to \{0, 1\}$, defined as $\mathcal{I}(t, m) = \mathbb{1}[t[A_i] = a_i \wedge t[A_j] = a_j]$. By storing $\mathcal{I}$ as a sparse index, we ensure $\mathcal{O}(1)$ lookup complexity during subsequent phases with minimal memory overhead.

Loss Calculation (Algorithm 3). As defined in Definition 6, the loss function reflects the weighted sum between the deletion ratio and the marginal error. At the hypothetical removal of tuples $t_1, \ldots, t_k$, the new size of the data would be $n_{new} = n_{cur} - k$, and the deletion ratio can be easily calculated by $\frac{n - n_{new}}{n}$. For marginal $m \in \mathcal{M}$ we can calculate the error as

$$\frac{\left|\frac{c_m - \sum_{i=1}^{k} \mathcal{I}(t_i, m)}{n_{new}} - \tilde{m}(a_i, a_j)\right|}{\tilde{m}(a_i, a_j)} \tag{7}$$

This is because $c_m - \mathcal{I}(t, m)$ represents the number of tuples in the data related to m after t was removed, thus $\frac{c_m - \mathcal{I}(t, m)}{n_{new}}$ represents the value of m in the data after the removal. This operation is repeated for all marginals and the results are averaged to get the average relative error. For each marginal we do an $\mathcal{O}(1)$ operation thus the complexity of the entire calculation of $\mathcal{L} \setminus \{t\}$ is $\mathcal{O}(|\mathcal{M}|)$.

State Update (Algorithm 4). Upon selecting tuples $t_1^*, \ldots, t_k^*$ for removal, we perform a localized update to the materialized state. For each $m \in \mathcal{M}$ we decrement $c_m \leftarrow c_m - \sum_{i=1}^{k} \mathcal{I}(t_i, m)$. The current dataset size is decremented: $n_{\text{cur}} \leftarrow n_{\text{cur}} - k$. Since $\tilde{m}(a_i, a_j)$ is fixed (a target given as input, not recomputed during repair), all the information needed for the next iteration is already given by this update. No full dataset scan is needed.

Complexity Analysis. The initialization (Algorithm 2) is a one-time cost of $\mathcal{O}(n \cdot |\mathcal{M}|)$. Crucially, for each subsequent iteration, the weight calculation (Algorithm 3) iterates over the marginals and computes a single arithmetic expression, running in $\mathcal{O}(|\mathcal{M}|)$ time. Note that we consider k as a constant. The state update (Algorithm 4) decrements the relevant counts and the dataset size, also in $\mathcal{O}(|\mathcal{M}|)$ time. Since $|\mathcal{M}|$ is typically small and independent of n, this incremental strategy to calculate the loss achieves near-constant time updates per tuple removal, ensuring high throughput for large-scale data cleaning tasks.

Algorithm 5: Batch Deletion

Input : Current dataset D_r, loss function $\mathcal{L}$, batch size k

Output: A removable batch $C^\star$

$C \leftarrow$ the k tuples with the smallest values of $\mathcal{L}(D_r \setminus \{t\})$, sorted in ascending order;

$\ell \leftarrow k$;

while $\ell > 1$ **do**

  if $\mathcal{L}(D_r \setminus C[1:\ell]) < \mathcal{L}(D_r)$ **then**

    return $C[1:\ell]$;

  $\ell \leftarrow \lceil \ell/2 \rceil$;

return $C[1]$;

Algorithm 5 replaces the single-tuple deletion performed in Lines 7–8 of Algorithm 1. It first ranks tuples according to their individual removal loss, $\mathcal{L}(D_r \setminus \{t\})$, and retains the k best candidates. Since the candidates are ordered by increasing loss, the algorithm repeatedly tests whether removing the current prefix decreases the objective. If so, the entire prefix is removed; otherwise, the prefix size is halved and tested again.

G. Pseudocode

Algorithm 6 presents the pseudocode of the private marginal selection and release procedure described in Section A.

H. Privacy and Utility Analysis

We next analyze Algorithm 6, showing that it satisfies zCDP and its error bound. We then prove the sensitivity of the utility function in Definition 17.

I. Proof of Proposition 16

Proof. **(1) Differential Privacy.** Algorithm 6 proceeds in two sequential stages: marginal selection (Lines 1–7) and noisy measurement computation (Line 11).

Algorithm 6: Obtain k representative noisy marginals

input : Private and synthetic datasets D_p, D_s, a utility function u, a privacy parameter for top-k ρ_{Topk}, a number of desired marginals k, and a privacy parameter for Gaussian marginal measurement ρ_{marg}.

output: The monitored set $\mathcal{M}$: for each of the k selected attribute pairs (A_i, A_j), all of its value-pair entries with their noisy estimates $\tilde{m}(A_i{=}a_i, A_j{=}a_j)$.

1 Compute all 2-way marginals of D_p and of D_s, $m_D(i,j)$;

2 Let $\sigma = 2\Delta_u\sqrt{\frac{k}{8\rho_{Topk}}}$, $\mathcal{M} = \emptyset$;

3 **foreach** *marginal* $m_{D_p}(i,j)$ **do**

4 $\quad score_u(m_{D_p}(i,j)) = u(m_{D_p}(i,j)) + Gumbel(\sigma)$;

5 **end**

6 Sort the marginals by $score_u$;

7 Let $\mathcal{T}_k$ be the top-k marginals by $score_u$;

/* Gaussian Mechanism: add $\mathcal{N}(0, \sigma^2_{\text{marg}})$ noise to each marginal entry (ρ_{marg}/k-zCDP per marginal). */

8 $\sigma_{\text{marg}} \leftarrow \dfrac{\sqrt{k}}{n\sqrt{\rho_{marg}}}$;

9 **foreach** $(A_i, A_j) \in \mathcal{T}_k$ **do**

10 $\quad$ **foreach** $(a_i, a_j) \in Dom(A_i) \times Dom(A_j)$ **do**

11 $\quad\quad \tilde{m}(A_i{=}a_i, A_j{=}a_j) \leftarrow m_{D_p}(A_i{=}a_i, A_j{=}a_j) + \mathcal{N}(0, \sigma^2_{\text{marg}})$;

12 $\quad$ **end**

13 $\quad$ Clip each entry to $[0,1]$ and renormalize so entries sum to 1;

14 $\quad \mathcal{M} \leftarrow \mathcal{M} \cup \{(A_i{=}a_i, A_j{=}a_j,\ \tilde{m}(A_i{=}a_i, A_j{=}a_j))\}$;

15 **end**

16 **return** $\mathcal{M}$;

The first stage selects k marginals using the Exponential Mechanism [55]. Following established results on the adaptive composition of the Exponential Mechanism [56], [58], [59], this sequence satisfies ρ_{Topk}-zCDP. We stress that this stage relies on the *bounded-range* analysis of the EM, not on the generic $\frac{1}{2}\epsilon^2$-zCDP bound recorded in Section A0a. An EM invocation with parameter ϵ is ϵ-bounded-range and therefore satisfies the strictly tighter $\frac{1}{8}\epsilon^2$-zCDP [58]–[60]; it is this $\frac{1}{8}$ constant, and not the $\frac{1}{2}$ constant of the preliminaries, that our accounting uses. Allocating ρ_{Topk}/k-zCDP to each of the k selections and inverting $\frac{1}{8}\epsilon^2 = \rho_{Topk}/k$ yields the per-step parameter $\epsilon = \sqrt{8\rho_{Topk}/k}$, which is exactly the value to which Algorithm 6 (Line 2) calibrates the Gumbel scale $\sigma = 2\Delta_u/\epsilon = 2\Delta_u\sqrt{k/(8\rho_{Topk})}$. Had we instead used the looser $\frac{1}{2}\epsilon^2$ bound of Section A0a, the per-step parameter would be $\epsilon = \sqrt{2\rho_{Topk}/k}$ and the calibrated Gumbel scale would be larger by a factor of 2 (i.e., twice the injected noise), degrading the utility bound in Part (2) accordingly. The second stage applies the Gaussian Mechanism to each of the k selected marginals with per-marginal budget ρ_{marg}/k-zCDP (noise $\sigma_{\text{marg}} = \sqrt{k}/(n\sqrt{\rho_{marg}})$, calibrated to the ℓ_2 sensitivity $\Delta_2 = \sqrt{2}/n$ of the vector of all entries of a normalized marginal: replacing one tuple changes at most two entries by $1/n$ each; see Definition 17). By sequential composition of the k independent Gaussian measurements (Proposition 15), this stage satisfies ρ_{marg}-zCDP in total. By the sequential composition property of zCDP (Proposition 15), Algorithm 6 is $(\rho_{Topk} + \rho_{marg})$-zCDP.

(2) Utility Bound. We derive the utility bound by extending the standard utility theorem for the Exponential Mechanism (EM) [55], as presented in [2]. Denote the exponential mechanism by EM. Recall that for a single execution with privacy budget ε, the probability that the EM outputs a result $r = EM(D, \mathcal{R})$ (with candidates set $\mathcal{R}$) with utility $u(D, r)$ significantly below the optimal utility OPT is bounded by:

$$\Pr\left[u(r) \le OPT - \frac{2\Delta_u}{\varepsilon}(\ln(|\mathcal{R}|) + t)\right] \le e^{-t} \tag{8}$$

where Δ_u is the sensitivity of the utility function. To adapt this to our setting of k selections over the candidates set of all the marginals $\mathcal{M}_0$, we define the effective per-step budget $\varepsilon = \sqrt{8\rho_{Topk}/k}$ (the bounded-range parameter derived in Part (1) by inverting $\frac{1}{8}\varepsilon^2 = \rho_{Topk}/k$; note this is the $\frac{1}{8}\varepsilon^2$-zCDP bound of the EM [58]–[60], not the $\frac{1}{2}\varepsilon^2$ bound of Section A0a).

Before applying this bound per step, we justify treating the one-shot selection of Algorithm 6 as k sequential EM steps. The algorithm perturbs every candidate's utility with a single draw of Gumbel noise and returns the k highest-scoring marginals (Lines 4–7). Because the Exponential Mechanism is equivalent to Report-Noisy-Max with Gumbel noise, this one-shot top-k has the same joint distribution as the sequential "peeling" process that runs k Exponential Mechanisms, where the i-th mechanism selects from the candidates not yet chosen [56]. Consequently, the marginal returned at index i is distributed exactly as the i-th sequential EM over the remaining candidates, so the single-execution tail bound above applies verbatim to each index.

Let $OPT^{(i)}$ denote the optimal utility available at the i-th step, and $\mathcal{M}_0^{(i)}$ the marginal selected at that step. Following the proof structure in [2], for any utility threshold s, the probability of selecting a candidate with utility at most s is bounded by:

$$\Pr\left[u(\mathcal{M}_0^{(i)}) \le s\right] \le \frac{|\mathcal{M}_0| \exp\left(\frac{\varepsilon s}{2\Delta_u}\right)}{\exp\left(\frac{\varepsilon OPT^{(i)}}{2\Delta_u}\right)} = |\mathcal{M}_0| \exp\left(\frac{\varepsilon}{2\Delta_u}(s - OPT^{(i)})\right) \tag{9}$$

Substituting $s = OPT^{(i)} - \frac{2\Delta_u}{\varepsilon}(\ln(|\mathcal{M}_0|) + t)$ into the inequality yields the desired bound for each step $i \in \{1, \ldots, k\}$. □

J. Proof of Proposition 18

Proof. Fix an attribute pair (A_i, A_j) and write $c_{D_p}(a_i, a_j) = |\{t \in D_p : t[A_i] = a_i \wedge t[A_j] = a_j\}|$ for the cell counts, so that $m_{D_p}(A_i{=}a_i, A_j{=}a_j) = c_{D_p}(a_i, a_j)/n$ with $n = |D_p|$. By Definition 17,

$$u(A_i, A_j) = \sum_{(a_i, a_j)} \left| \tfrac{c_{D_p}(a_i, a_j)}{n} - m_{D_s}(A_i{=}a_i, A_j{=}a_j) \right|.$$

Let D_p and D_p' be neighboring private datasets (Section II-B), i.e., D_p' is obtained from D_p by replacing one tuple t with a tuple t'; the synthetic marginals $m_{D_s}(\cdot,\cdot)$ and the size n

TABLE V: Full list of denial constraints per dataset used in Section V.

Dataset	Constraints
IPUMS-CPS	(1) $\forall t_1, t_2 \in D : \neg(t_1.AGEP = 0 \wedge t_2.AGEP = 0 \wedge t_1.SCHL \neq t_2.SCHL)$ (2) $\forall t_1, t_2 \in D : \neg(t_1.AGEP = 0 \wedge t_2.AGEP = 0 \wedge t_1.WAGP \neq t_2.WAGP)$ (3) $\forall t_1, t_2 \in D : \neg(t_1.CIT = t_2.CIT \wedge t_1.NATIVITY \neq t_2.NATIVITY)$ (4) $\forall t_1, t_2 \in D : \neg(t_1.AGEP = 0 \wedge t_2.AGEP = 0 \wedge t_1.MAR \neq t_2.MAR)$ (5) $\forall t_1, t_2 \in D : \neg(t_1.ST = t_2.ST \wedge t_1.DIVISION \neq t_2.DIVISION)$
Adult	(1) $\forall t_1, t_2 \in D : \neg(t_1.education_num \neq t_2.education_num \wedge t_1.education = t_2.education)$ (2) $\forall t_1, t_2 \in D : \neg(t_1.education_num = t_2.education_num \wedge t_1.education \neq t_2.education)$ (3) $\forall t_1, t_2 \in D : \neg(t_1.capital_gain < t_2.capital_gain \wedge t_1.capital_loss < t_2.capital_loss)$
Compas	(1) $\forall t_1, t_2 \in D : \neg(t_1.DecileScore = t_2.DecileScore \wedge t_1.ScoreText \neq t_2.ScoreText)$ (2) $\forall t_1, t_2 \in D : \neg(t_1.Scale_ID = t_2.Scale_ID \wedge t_1.DisplayText \neq t_2.DisplayText)$ (3) $\forall t_1, t_2 \in D : \neg(t_1.RecSupervisionLevel = t_2.RecSupervisionLevel \wedge t_1.RecSupervisionLevelText \neq t_2.RecSupervisionLevelText)$ (4) $\forall t_1, t_2 \in D : \neg(t_1.DisplayText = t_2.DisplayText \wedge t_1.Scale_ID \neq t_2.Scale_ID)$ (5) $\forall t_1, t_2 \in D : \neg(t_1.RecSupervisionLevelText = t_2.RecSupervisionLevelText \wedge t_1.RecSupervisionLevel \neq t_2.RecSupervisionLevel)$
Tax	(1) $\forall t_1, t_2 \in D : \neg(t_1.State = t_2.State \wedge t_1.HasChild = t_2.HasChild \wedge t_1.ChildExemp \neq t_2.ChildExemp)$ (2) $\forall t_1, t_2 \in D : \neg(t_1.State = t_2.State \wedge t_1.MaritalStatus = t_2.MaritalStatus \wedge t_1.SingleExemp \neq t_2.SingleExemp)$ (3) $\forall t_1, t_2 \in D : \neg(t_1.State = t_2.State \wedge t_1.Salary > t_2.Salary \wedge t_1.Rate < t_2.Rate)$ (4) $\forall t_1, t_2 \in D : \neg(t_1.State = t_2.State \wedge t_1.Salary < t_2.Salary \wedge t_1.Rate > t_2.Rate)$

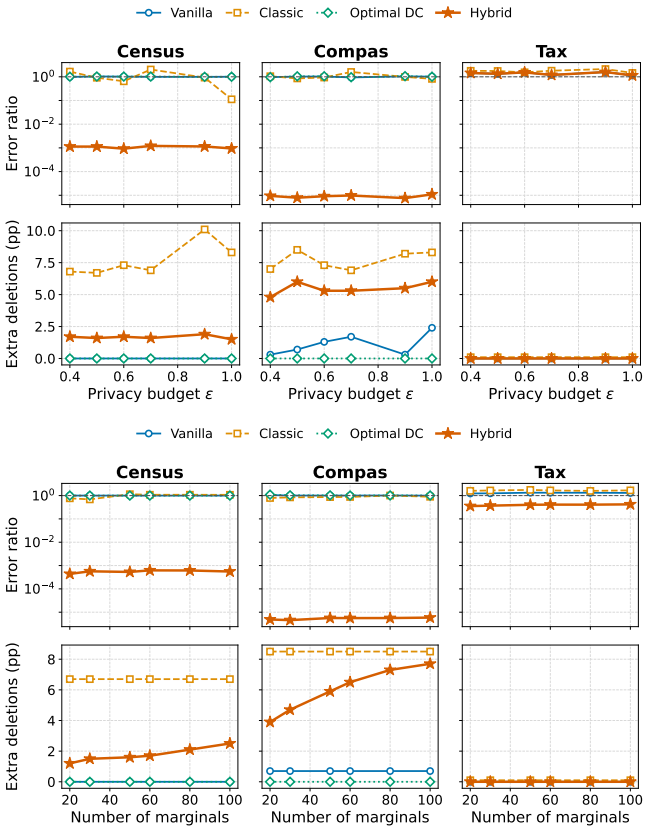


Fig. 9: Sensitivity of repair quality for PATE-CTGAN, as in Figure 3.

are unchanged. Applying the triangle inequality $\big||x| - |y|\big| \leq |x - y|$ to each cell and summing,

$$|u_{D_p}(A_i, A_j) - u_{D'_p}(A_i, A_j)| \leq \frac{1}{n} \sum_{(a_i, a_j)} \big|c_{D_p}(a_i, a_j) - c_{D'_p}(a_i, a_j)\big|.$$

Replacing t with t' decreases the count of the cell $(t[A_i], t[A_j])$ by one and increases the count of the cell $(t'[A_i], t'[A_j])$ by one, changing no other cell, so the sum on the right-hand side is at most 2. Hence $\Delta_u \leq 2/n$, and the bound is attained when t and t' fall in different cells and both cell differences push u in the same direction. Therefore $\Delta_u = \frac{2}{n}$. □

The full set of DCs per dataset appears in Table V. Most of the DCs are taken from previous work [13], [19], [38] and express innate semantic dependencies between attributes in the datasets. Some new constraints were discovered due to bucketization. Specifically, for [22], we bucketize AGEP and WAGP which represent the age and income of a person. Thus, for example, the first DC in Table V expresses that two tuples in a specific age group (coded by the bucket 0) have the same years of schooling. This constraint was examined and found to be satisfied by the data.

Figures 8 to 10 repeat Figure 3 of the main text for the remaining generators. Figures 11a, 11b, 12a and 12b report

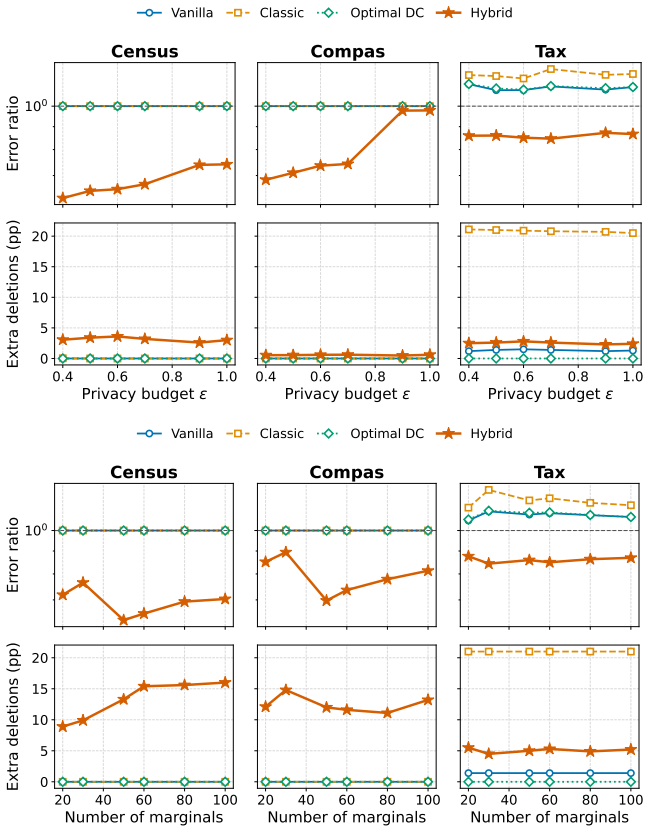


Fig. 10: Sensitivity of repair quality for Kamino (5K tuples), as in Figure 3.

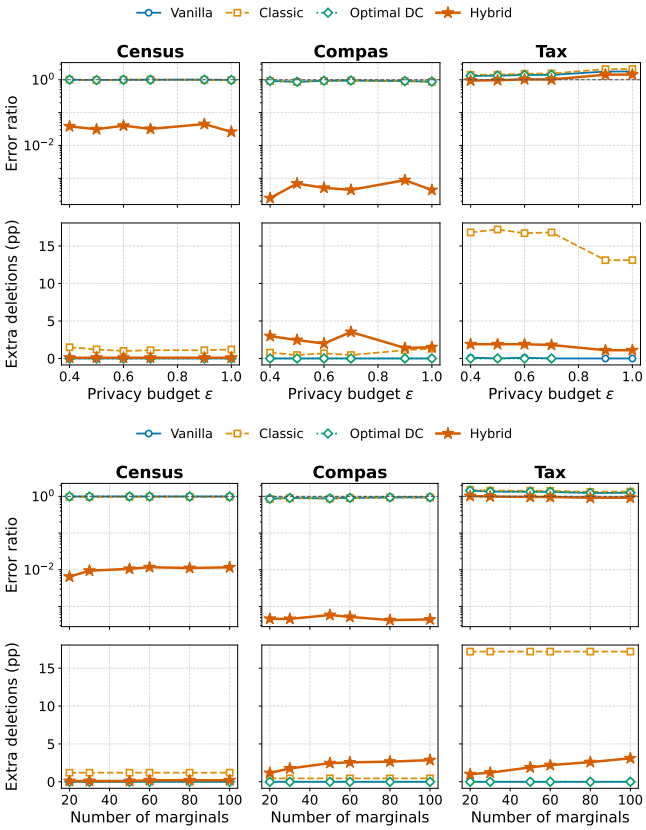
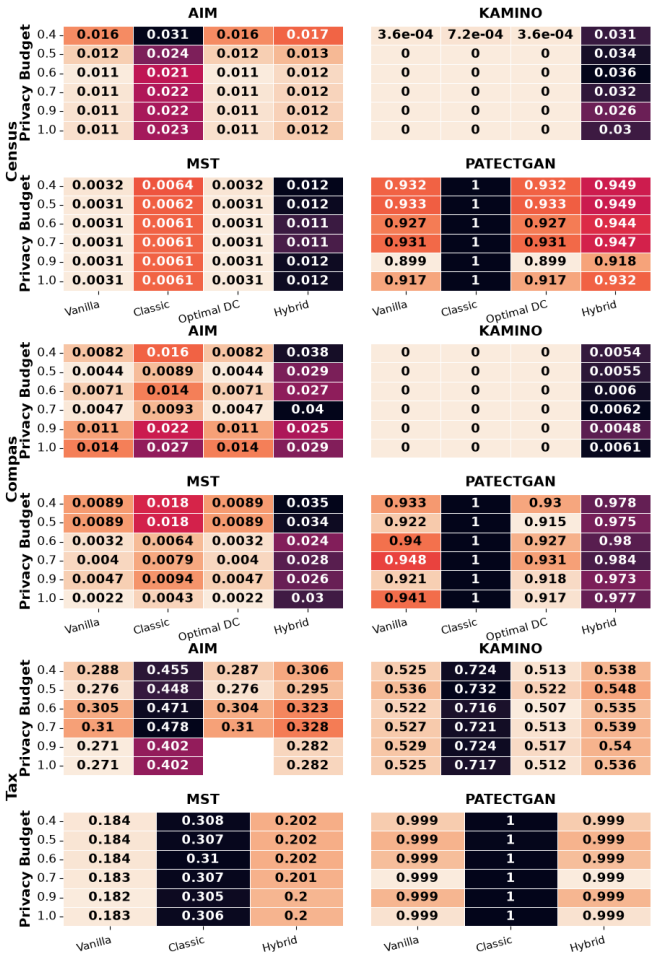
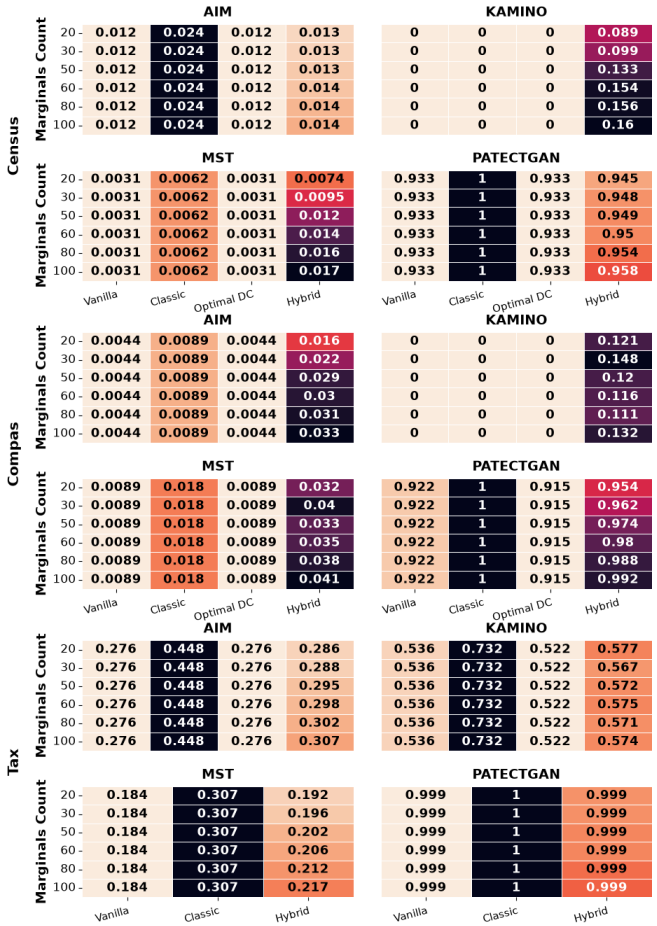


Fig. 8: Sensitivity of repair quality for AIM (top: varying ϵ; bottom: varying the number of marginals), as in Figure 3.

the absolute deletion ratio and marginal error for every dataset, generator, and repair method underlying Figures 2 and 3. Empty cells indicate that OPTIMAL-DC-BASELINE reached the 12-hour timeout.

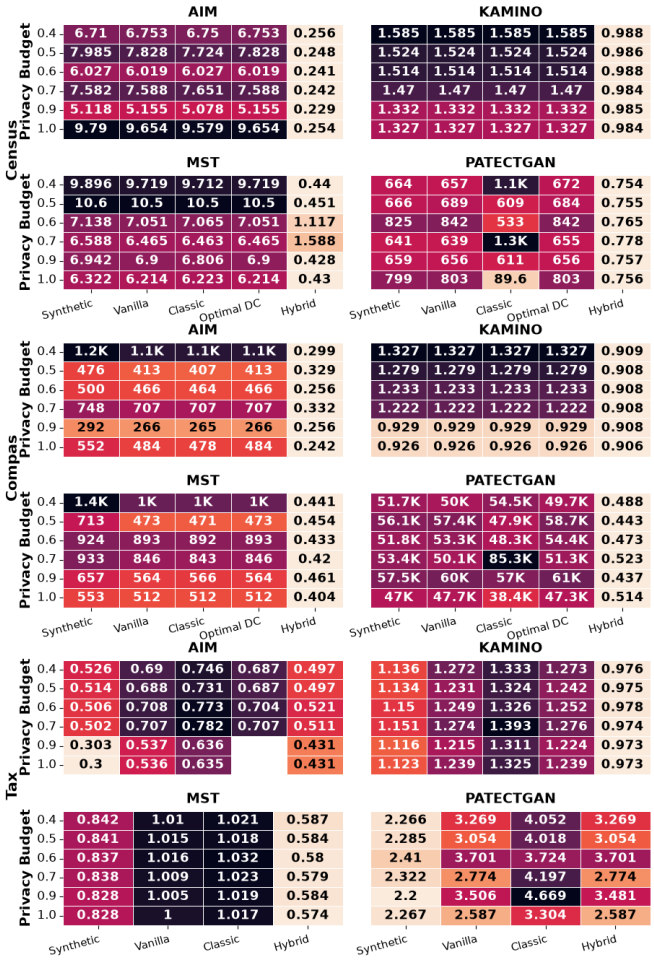


(a) Deletion ratio as a function of the generation privacy budget ϵ.

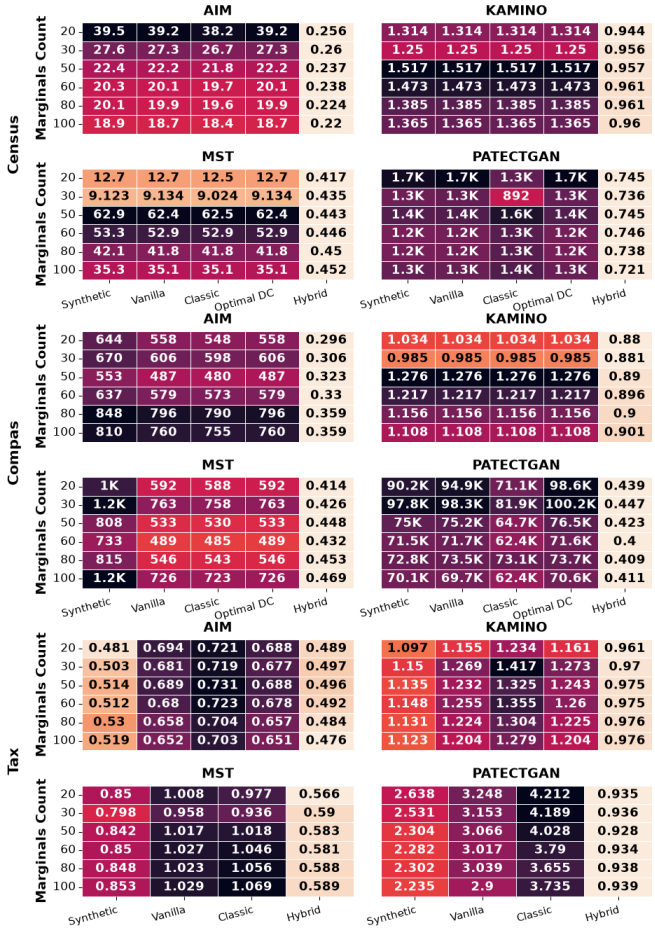


(b) Deletion ratio as a function of the number of marginals.

Fig. 11: Per-configuration deletion ratio of every repair method. Empty cells indicate that OPTIMAL-DC-BASELINE reached the 12-hour timeout.



(a) Marginal error as a function of the generation privacy budget ϵ.



(b) Marginal error as a function of the number of marginals.

Fig. 12: Per-configuration marginal error of every repair method and of the unrepaired synthetic data.

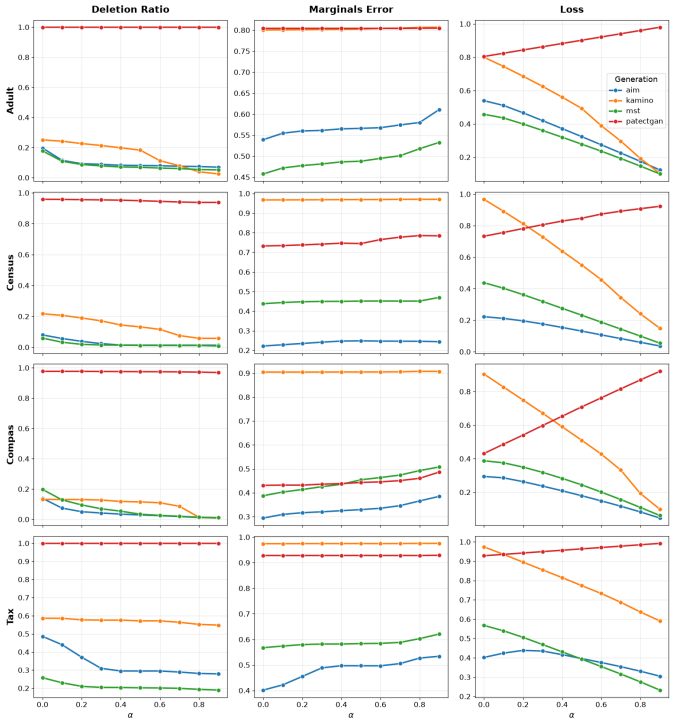


Fig. 13: Deletion ratio (left) marginal error (center) and loss (right) of HYBRID as function of the tradeoff parameter α.

Figure 13 reports the deletion ratio (left), mean marginal error (center), and optimization loss (right) of HYBRIDas a function of the tradeoff parameter α. As expected, increasing α consistently decreases the deletion ratio while slightly increasing the marginal error. This follows directly from the loss function introduced in Definition 6,

$$\mathcal{L}(D_r) = \alpha \frac{n - |D_r|}{n} + (1-\alpha)\frac{1}{|\mathcal{M}|}\sum_{m\in\mathcal{M}} E_m(m, D_r),$$

where $E_m(m, D_r)$ denotes the relative error of marginal m on the repaired dataset D_r. Larger values of α place more emphasis on preserving tuples and less emphasis on improving the marginal statistics.

This behavior can also be seen from the acceptance criterion of the repair algorithm. Since every accepted deletion must decrease the loss, if removing k tuples transforms $D_r^{(i)}$ into $D_r^{(i+1)}$, then

$$\mathcal{L}(D_r^{(i+1)}) < \mathcal{L}(D_r^{(i)}).$$

Rearranging the loss yields

$$\frac{1}{|\mathcal{M}|}\sum_{m\in\mathcal{M}}\left(E_m(m, D_r^{(i)}) - E_m(m, D_r^{(i+1)})\right) > \frac{\alpha}{1-\alpha}\cdot\frac{k}{n}$$

Therefore, every deletion must improve the average marginal error by at least $\frac{\alpha}{1-\alpha}\cdot\frac{k}{n}$. Since this threshold is monotonically increasing in α, larger values of α make deletions progressively harder to justify, resulting in fewer removed tuples and slightly larger marginal error.

The extreme case $\alpha = 1$ corresponds exactly to the classical repair objective,

$$\mathcal{L}(D_r) = \frac{n - |D_r|}{n},$$

which ignores marginal quality altogether. Consequently, after the first phase establishes a feasible repair, no further deletion can reduce the objective, and the marginal-improvement phase terminates immediately.

Empirically, however, the effect of α is relatively modest across all datasets. Although the threshold $\frac{\alpha}{1-\alpha}\frac{k}{n}$ grows rapidly as α approaches one, in practice $k \ll n$, so the required improvement in marginal error remains small over the range of α considered in our experiments. As a result, the deletion ratio and marginal error vary only gradually.

The optimization loss changes approximately linearly with α, which is expected because both the deletion ratio and the marginal error remain nearly constant throughout the sweep. In this regime, the loss is approximately an affine function of α with slope

$$\frac{n - |D_r|}{n} - \frac{1}{|\mathcal{M}|}\sum_{m\in\mathcal{M}} E_m(m, D_r).$$

For PATE-CTGAN, the deletion ratio is substantially larger due to the large number of constraint violations, yielding a positive slope and an increasing loss. In contrast, AIM, MST, and Kamino exhibit much lower deletion ratios, resulting in a negative slope and a decreasing loss as α increases.

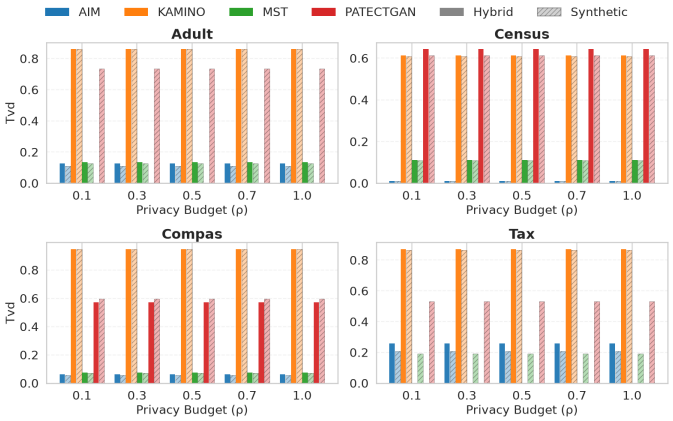


Fig. 14: TVD of repaired data using HYBRID as function of privacy budget given to the marginals.

Figure 14 reports the two-way TVD with respect to the private data as a function of the privacy budget allocated to marginal estimation, complementing the marginal-error plot of Figure 7 in the main text. As described in Section V-A, the target marginals are selected uniformly at random and estimated using the Gaussian mechanism. These experiments evaluate how the accuracy of the noisy marginals affects the quality of the repaired dataset.

As expected, repair slightly increases the two-way TVD compared to the original synthetic data. This is because the optimization explicitly targets the selected marginals rather than the full joint distribution, so improving the selected marginals may slightly degrade global distributional similarity. Nevertheless, the increase is small and remains largely insensitive to the privacy budget. This is expected since only a subset of marginals is optimized, and improvements to their estimates have a limited effect on the overall TVD.

In contrast, the marginal error consistently decreases as the privacy budget increases. With a larger budget, the Gaussian mechanism adds less noise to the target marginals, providing more accurate optimization targets and allowing the repair algorithm to better match the corresponding statistics of the private data.